# Confidence vs. Competence: Misalignment in Judgment and Performance for Agentic Software Repair

MINGYUE YUAN, University of New South Wales & CSIRO, Australia
JIESHAN CHEN*, CSIRO, Australia
DEHAI ZHAO, Zhejiang University, China
GELAREH MOHAMMADI, University of New South Wales, Australia
YOSHIFUMI KITAMURA, Tohoku University, Japan
AARON JOHN QUIGLEY, The Australian National University, Australia
QINGHUA LU, CSIRO, Australia
ZHENCHANG XING, CSIRO, Australia

While Large Language Model (LLM) agents are increasingly applied to automated software repair, misalignment remains in how humans and agents judge issue conditions and how agents' pre-execution self-assessments relate to repair competence. Human developers rely on diagnostic cues such as reproduction steps and stack traces to judge whether an issue is sufficiently specified, whereas LLM agents often fail to recognize missing information. We present the first systematic empirical study of misalignment between human and agent judgments and between agent judgments and repair performance. Using SWE-bench, controlled ablation experiments establish a causal link between removing human-valued cues and reduced LLM repair success. Specifically, LLM judges show limited agreement with human problem-specification ratings, and agents' pre-execution self-assessments only weakly track repair degradation when key cues are removed. Our trajectory analysis further reveals distinct behavioral responses to missing information, while post-execution self-judgment signals add useful discriminative information when combined with behavioral traces. These findings reveal a persistent gap between how current repair agents assess issue conditions and how they perform during repair, highlighting the need for judgment-aware and correction-aware agent design.

CCS Concepts: • **Software and its engineering** → Software testing and debugging; Software evolution; • **Computing methodologies** → Artificial intelligence.

Additional Key Words and Phrases: Large Language Models, LLM Agents, Software Repair, Empirical Study, LLM Self-Judgment

## 1 Introduction

Rapid advancement of large language models (LLMs) has enabled significant progress in automated software engineering [3, 25, 27, 39, 41], enabling tasks such as code generation [38, 51], bug report analysis [7, 15, 19], and program repair [8, 11, 50]. Recent agent-based systems, such as OpenHands [45], RepoMaster [44], and SWE-agent [48], demonstrate promising capabilities in addressing real-world GitHub issues. Despite these advances, LLM agents often struggle with real-world ambiguity and yield inconsistent performance. Prior research has examined these shortcomings from system-level perspectives, attributing failures to architectural limitations [9, 53], overthinking behaviors [13], or dataset artifacts such as solution leakage and weak test cases [2]. However, these analyses overlook a critical factor: the potential misalignment in judgment between LLM agents and human developers.

*Corresponding author.

Authors' Contact Information: Mingyue Yuan, University of New South Wales & CSIRO, Australia, mingyue.yuan@unsw.edu.au; Jieshan Chen, CSIRO, Australia, jieshan.chen@csiro.au; Dehai Zhao, Zhejiang University, China, dehai.zhao@zju.edu.cn; Gelareh Mohammadi, University of New South Wales, Australia, g.mohammadi@unsw.edu.au; Yoshifumi Kitamura, Tohoku University, Japan, kitamura@riec.tohoku.ac.jp; Aaron John Quigley, The Australian National University, Australia, aquigley@gmail.com; Qinghua Lu, CSIRO, Australia, qinghua.lu@csiro.au; Zhenchang Xing, CSIRO, Australia, zhenchang.xing@csiro.au.

Unlike LLM agents that tend to approach diverse tasks with a uniform trial-and-error strategy, human engineers perform an initial triage before selecting a problem-solving approach. They routinely assess issue reports using diagnostic cues to judge whether a problem is sufficiently well-defined for resolution, such as reproduction steps, stack traces, and error messages [5, 10]. Recent research has attempted to emulate this human-like flexibility by assigning modular roles or dynamically constructing reasoning workflows. Systems like MarsCode Agents and Atomic Reasoner [30, 32] delegate responsibilities to specialized roles (such as searcher, programmer, tester) or employ cognitive routing mechanisms to assemble reasoning pathways. Workflow generators such as AFlow [52] and DARSagent [1] automate multi-agent collaboration through evolutionary search. However, these approaches largely overlook a foundational factor, i.e., the possibility of underlying differences in judgment between LLMs and humans in how they interpret and prioritize information in issue reports. Instead of questioning whether the agent truly understands what makes a problem solvable, most systems assume that resolution is always possible and focus on optimizing execution [9]. This blind spot leads to brittle behavior in ambiguous or underspecified scenarios, where LLMs frequently overestimate their capabilities, waste resources, and fail inconsistently [53]. This motivates a fundamental shift in perspective: before improving workflows or capabilities, we must first understand how LLM agents perceive problem difficulty and quality. *Do they recognize whether an issue is solvable? How does their judgment compare to that of human developers? And how does this misalignment affect downstream agent behavior and performance?*

To investigate these questions, we present the first empirical study of the **two judgment-related gaps in agentic software repair: a human–agent judgment gap, where LLM judgments of issue conditions differ from human judgments, and a confidence–competence gap, concerning whether agents' pre-execution self-assessments track downstream repair success.**

Our study is organized around four complementary research questions. **RQ1** studies *Problem Specification Informativeness and Judgment Alignment*, asking how informative initial problem statements and subsequent developer hints are for assessing issue specification, and to what extent LLM judgments align with human judgments of specification and difficulty. **RQ2** studies *Pre-execution Confidence–Competence Alignment*, asking whether, under controlled cue perturbations, changes in agents' pre-execution self-assessments track corresponding changes in repair success. **RQ3** studies *Behavioral Response to Input Perturbations*, asking how repair trajectories change when diagnostic cues are removed or rephrased and whether those trajectory changes correspond to productive or unproductive behavioral response. **RQ4** studies *Toward Self-Judgment-Based Correction*, asking whether post-execution self-judgment and observable trajectory evidence can detect likely failed attempts as complementary validation signals for correction.

Specifically, our analysis is grounded in 2,294 issue reports from the SWE-bench benchmark [21]. For RQ1, we first enrich the dataset with human-annotated diagnostic cues and use OpenAI-provided problem-specification and difficulty ratings [36] to characterize issue-report informativeness from the human side. We then introduce an LLM-as-judge module to elicit agents' explicit assessments of issue clarity and difficulty, and examine how the human annotations align with two agent-side signals: explicit LLM judgments and implicit uncertainty signals [29, 31]. This comparison reveals a human–LLM judgment alignment gap in assessing issue-report informativeness. To evaluate the consequences of this misalignment, we design a series of controlled ablation experiments (RQ2) in which we selectively remove or rephrase key diagnostic cues from issue reports. We observe a significant drop in success rate (up to 8.9%) when human-valued information is removed, while changes in agents' pre-execution self-assessments and implicit uncertainty signals only partially reflect this loss of diagnostic content. We further trace agent behavioral response using a trajectory operation tree, extracting interpretable features that reflect how agents respond when faced with

incomplete or ambiguous inputs (RQ3). This reveals measurable but generally modest behavioral shifts, such as changes in execution effort and verification behavior, and allows us to distinguish productive response patterns from longer but less effective trajectories. Finally, we assess whether agents can reflect on their completed repair process to detect likely failed attempts. By leveraging post-execution self-judgment signals and behavioral features, we show that post-execution self-judgment is weak as a stand-alone signal but adds complementary discriminative information when combined with behavioral traces (RQ4), supporting its use as an auxiliary validation signal for deciding whether a candidate patch should be accepted, further validated, or revised. We release all code, annotations, and experimental scripts at https://github.com/Mingyue-eva/AgentCognitive.

In conclusion, our contribution is as follows:

- We present the first empirical study of **two judgment-related gaps** in agentic software repair: the gap between human and LLM judgments of issue conditions, and the gap between agents' pre-execution self-assessments and downstream repair competence.
- We propose a **layered analysis framework** with trajectory operation trees based on controlled diagnostic-cue perturbation, enabling us to examine both how human-valued diagnostic cues affect repair success and how agents' behavioral response patterns change when such cues are removed or rephrased.
- We take a first step toward **self-judgment-based correction** by showing that post-execution self-judgment features provide complementary discriminative information when combined with behavioral traces for detecting likely repair failure.

## 2 Related Work

**Empirical and Analysis Studies of Software Repair:** In the software engineering context, Bettenburg et al. [5] identified key components of effective bug reports through developer surveys, including observed behavior, expected behavior, and steps to reproduce. Chaparro et al. [10] further showed that missing expected behavior or reproduction steps can substantially delay bug triage. Fan et al. [14] further analyze the distinction between valid and invalid bug reports. Wang et al. [46] propose a taxonomy of LLM code-generation errors.

In addition, SWE-bench+ [2] reveals critical flaws in "solution leakage" and weak test cases that inflate LLM repair rates. Cemri et al. [9] and Zhang et al. [53] classify multi-agent failures into design misalignments and verification gaps. Cuadron et al. [13] identify "overthinking" in reasoning agents. On the human side, Kaur et al. [24] demonstrate that agent–human pairs can match productivity but often fall short in creative framing and planning. Gonçalves et al. [17] extend Letovsky's model to analyze code review comprehension strategies. However, they do not jointly analyze how human-valued diagnostic cues shape both human judgments and agent behavior under controlled perturbation.

**LLM Agents for Software Repair:** Recent advances cast LLMs as autonomous software repair agents, ranging from fully automatic systems like SWE-agent [48] and OpenHands [45], which generate patches and execute tests but lack planning and may suffer "overthinking" loops [13], to enhanced reasoning extensions [54] that still omit systematic self-evaluation. More structured routing frameworks—e.g., MarsCode agents and Atomic Reasoner [30, 32], which divide the repair process into roles such as Searcher, Programmer, and Tester, may be fragile under ambiguous inputs. Emerging workflow generators [1, 52] automate multi-agent pipelines via evolutionary search, but these approaches rely on costly validation over predefined sets to select optimal workflows and produce pipelines that remain fixed. Together, these approaches demonstrate the technical breadth of LLM-based repair. However, these approaches largely do not examine whether agent judgments

of issue conditions align with human judgments, or whether agents' pre-execution self-assessments align with downstream repair outcomes.

**Self-Assessment and Uncertainty in LLM Agents:** Recent studies demonstrate that LLMs often fail to be calibrated, exhibiting overconfidence that undermines reliability in critical tasks [22]. Although social, factual, and reasoning biases in LLMs have been extensively studied [4, 28, 33], their implications for software repair remain underexplored. Traditional uncertainty methods (Monte Carlo dropout, ensembles) do not scale to large generative models [16, 26], leading to black-box measures such as semantic uncertainty and token eccentricity [29, 31, 47] for implicit uncertainty signal estimation, and LLM-as-judge methods [1, 9, 13] for explicit assessment. In this work, we combine implicit uncertainty estimates with our "LLM-as-judge" assessments to analyze pre-execution self-assessment and post-execution self-judgment signals in software repair and their relationship to controlled input perturbations, repair outcomes, and trajectory-level behavior.

## 3 Methodology

We organize the methodology to make the empirical study logic explicit before introducing the operational details. We first describe the dataset, annotation labels, and task-level terminology used throughout the study, including *Problem Statement*, *Developer Hints*, *Gold Patch*, and *Test Patch*. We then present the four research questions that structure our empirical analysis. Finally, we introduce our conceptual framework and operationalization, followed by the specific methodological design for each research question and the implementation details.

### 3.1 Dataset

Our empirical study is based on SWE-bench [21], a large-scale benchmark containing 2,294 real-world GitHub issues from 12 popular Python repositories. This benchmark enables us to evaluate LLM agents on a realistic and important software engineering task, where each task corresponds to a single GitHub issue together with its repository context, reference patch, and evaluation tests. The input consists of the issue's problem statement and the full codebase, and the goal is to generate a patch that correctly resolves the issue. For each issue in SWE-bench, the benchmark provides *Metadata* with basic information about the issue, the user-reported *Problem Statement*, and the pull request, which contains a *Gold Patch* proposed by developers to fix the issue and a *Test Patch* to verify the fix. In some cases, before submitting the pull request, developers ask follow-up questions or provide explanations about the intended fix. We refer to these interactions as *Developer Hints*. An example is shown in Figure 1. We use "issue report" and "problem statement" interchangeably throughout this paper.

To ensure quality, OpenAI randomly selected 1,699 SWE-bench samples and annotated each with labels from three independent annotators [36]. In detail, each annotator is instructed to annotate each issue from multiple dimensions. The three key annotation dimensions are: (1) **Problem Specification:** Clarity and completeness of the problem statement. Rated on a 4-point scale (0–3), where 0–1 indicates minor underspecification and 2–3 indicates severe underspecification. (2) **Task Difficulty:** Estimated resolution time for an experienced software engineer – easy (<15 minutes), moderate (15 minutes–1 hour), hard (1–4 hours), very hard (>4 hours). In the subsequent analysis, these four ordinal categories are encoded as 0-3, respectively. (3) **Confidence Level:** Annotators' confidence in their judgment, rated on a 1–5 scale. We do not use this confidence rating in our experiments, as it is inherently subjective and may vary substantially across annotators. Instead, we focus on the more operationally grounded *Problem Specification* and *Task Difficulty* dimensions to characterize judgments.


Fig. 1. An example from SWE-bench. The Problem Statement describes the issue linked to a pull request (PR). The Test Patch defines the test case validating the fix, while the Gold Patch shows the actual code change (red: deletions, green: additions). Developer Hints are follow-up comments from developers.

The final label for each sample is the highest-severity label among the three annotators. Based on these annotations, OpenAI selected 500 tasks, named *SWE-bench Verified*, verified to be non-problematic and solvable. Our empirical study uses both the fully annotated set (**SWE-bench-1699**) and the high-quality **SWE-bench Verified** set.

We further partitioned the **SWE-bench-1699** dataset into **SWE-bench-1699-Specified** (n=1048) and **SWE-bench-1699-Underspecified** (n=648), following OpenAI's criteria for distinguishing high-quality and low-quality issue reports. The **SWE-bench-1699-Specified** set is a superset of **SWE-bench Verified**. We report results on both sets because they serve complementary purposes: **SWE-bench Verified** is the widely used public benchmark for repair-agent evaluation, while **SWE-bench-1699-Specified** provides a larger annotated set for checking whether the observed patterns are affected by the Verified-set selection.

These human-validated annotations enable detailed analysis of both human and LLM judgments, facilitating a direct comparison of how each perceives and addresses software issues.

### 3.2 Research Questions

We conduct an empirical study to examine how LLM agents interpret and respond to software issue information under controlled information conditions. We first present the four research questions that organize the study.

**RQ1: Problem Specification Informativeness and Judgment Alignment.** How informative are the *Problem Statement* and subsequent *Developer Hints* for assessing issue specification, and do LLM judgments align with human ratings and human-valued diagnostic cues?
**RQ2: Pre-execution Confidence–Competence Alignment.** Under controlled cue perturbations, do changes in an agent's pre-execution self-assessment track corresponding changes in repair success?
**RQ3: Behavioral Response to Input Perturbations.** How do repair trajectories change when diagnostic cues are removed or rephrased, and do these changes correspond to productive or unproductive behavioral response?
**RQ4: Toward Self-Judgment-Based Correction.** Can post-execution self-judgment and observable trajectory evidence discriminate successful from failed repair attempts as complementary validation signals?

### 3.3 Conceptual Framework and Operationalization

After presenting the research questions, we explain how they are grounded and measured in this section. We first clarify our functionalist stance toward LLM-agent behavior, focusing on observable behavioral representations under controlled information conditions rather than unobservable internal mental states. We then connect the four research questions to a problem-solving decomposition of software issue resolution: cue use, calibration, control, and monitoring. Finally, we summarize how each research question maps to its theoretical perspective and operational measures.
**Conceptual stance.** Our study does not assume that LLMs possess consciousness or human-like internal mental states. Instead, we adopt a functionalist stance and evaluate **observable behavioral representations** of LLM agents under controlled information conditions. Accordingly, references to human problem solving and cognitive theory are used as *theoretical anchors* for designing research questions and operational measures.
**Theoretical Motivation for the Research Questions.** Problem solving requires more than producing a final outcome for any problem solver [40]. To act effectively in an uncertain software issue resolution environment, a problem solver must (1) recognize whether the available report is diagnostically sufficient, (2) form a calibrated assessment of whether action is likely to succeed, (3) adjust its behavior as the task unfolds, and (4) monitor available signals to distinguish progress from likely failure, in order to decide whether to terminate execution or continue attempting resolution. Therefore, merely studying final fix success is incomplete: it cannot reveal whether failure is due to insufficient use of cues, poor self-assessment, ineffective behavioral response, or limited ability to identify failure-prone operations. Motivated by problem-solving and cognitive theory, we organize our empirical study around four corresponding dimensions: *Cues and Heuristics*, *Calibration*, *Control*, and *Monitoring*, as summarized in Table 1. These dimensions provide a principled decomposition of issue-resolution behavior and map naturally to different stages of an LLM agent's workflow.

Specifically, **RQ1** corresponds to **Cues and Heuristics** and studies **Problem Specification Informativeness and Judgment Alignment**. It asks how *Problem Statements* and *Developer Hints* convey information relevant to assessing issue specification, and whether LLM judgments align with human judgments of specification and difficulty. This question is grounded in cue-utilization approaches to metacognitive judgment [23]. It is operationalized in two parts: first, an information-theoretic view of uncertainty reduction using information-gain analysis inspired by recent work on reasoning efficiency [49]; and second, human–LLM judgment alignment analysis, where rank-based association is measured using Spearman correlation following prior software engineering practice [20].

**RQ2** corresponds to **Calibration** and studies **Pre-execution Confidence–Competence Alignment**. Under controlled cue perturbations, it asks whether changes in an agent's pre-execution

| RQ | Theoretical Perspective | Operational Measure |
|---|---|---|
| RQ1 | **Cues and Heuristics:** cue utilization and judgment [23] | **Problem Specification Informativeness and Judgment Alignment:** static cue association, dynamic developer follow-up, and information-gain analysis of *Problem Statements* and *Developer Hints* (§3.4.3), together with human–LLM judgment alignment analysis using explicit rating distributions, implicit uncertainty signals, and Spearman correlation (§3.4.5). |
| RQ2 | **Calibration:** monitoring and self-assessment accuracy [34] | **Pre-execution Confidence–Competence Alignment:** change-based alignment analysis between perturbation-induced shifts in pre-execution self-assessment and corresponding shifts in repair success, using Spearman correlation (§3.5.1). |
| RQ3 | **Control:** strategy adjustment under changing information conditions [35] | **Behavioral Response to Input Perturbations:** trajectory-tree construction, behavioral feature extraction, and attribution analysis of repair trajectories under cue removal and rephrasing (§3.6.1–§3.6.3). |
| RQ4 | **Monitoring:** signal detection and discriminability [18] | **Toward Self-Judgment-Based Correction:** failure-detection and signal-discriminability analysis using post-execution self-judgment and observable trajectory evidence for completed repair attempts (§3.7.3). |

Table 1. Mapping each research question to its theoretical perspective and corresponding operational measure. The four RQs also align with successive problem-solving stages in agentic software repair: RQ1 focuses on recognizing input sufficiency, RQ2 on forming a calibrated pre-execution self-assessment of whether action is likely to succeed, RQ3 on observable behavioral responses as the task progresses under perturbed inputs, and RQ4 on post-execution monitoring signals for failure detection and correction.

self-assessment track corresponding changes in repair success. This question is grounded in theories of monitoring and self-assessment accuracy [34] and is operationalized through change-based alignment analysis, using Spearman correlation to quantify whether shifts in *Problem Specification* and *Task Difficulty* ratings track perturbation-induced repair degradation.

**RQ3** corresponds to **Control** and studies **Behavioral Response to Input Perturbations**. It asks how repair trajectories change when diagnostic cues are removed or rephrased. This question follows information-processing and strategy-adjustment perspectives [35] and is operationalized through trajectory-tree construction, behavioral feature extraction, and interpretable attribution analysis.

Finally, **RQ4** corresponds to **Monitoring** and studies **Toward Self-Judgment-Based Correction**. It asks whether post-execution self-judgment and observable trajectory evidence can detect likely failed repair attempts and support subsequent corrective decision-making. This question is grounded in signal detection theory [18] and is operationalized through failure-detection analysis for completed repair attempts, following prior software engineering work on failure-related signal analysis [14].

### 3.4 RQ1: Problem Specification Informativeness and Judgment Alignment

We answer RQ1 from two perspectives. First, from the human side, we characterize issue-report informativeness in the human-labeled data through static cue association, dynamic developer follow-up, and information-gain analyses, which together explain why static, dynamic, and aggregate informativeness are all needed to assess problem specification. Second, from the LLM side, we test whether LLMs' explicit judgments and implicit uncertainty signals capture the same specification-relevant information as human annotations.

To investigate whether LLMs and human annotators differ in their judgments of issue-report informativeness and problem specification, we first annotated the SWE-bench dataset with diagnostic features commonly used by developers. In this study, we distinguish between the initial *Problem*

*Statement*, which refers to the user-reported issue description, and *Developer Hints*, which refer to subsequent diagnostic information provided in developer/maintainer follow-up comments when available. This annotated dataset enabled us to empirically examine which cues are associated with human judgments in both static and dynamic contexts, and the extent to which LLM judgments align with human judgments under the same information conditions.

*3.4.1 Annotating Key Diagnostic Features.* Given an issue report, experienced developers typically first assess whether the problem description contains sufficient information. This preliminary judgment, whether the task is tractable or ill-posed, guides their subsequent reasoning and repair strategies. Prior research [14] has shown that the perceived completeness of an issue report is often associated with the presence of certain diagnostic cues, such as reproduction steps and stack traces. These cues are not strictly necessary for a report to be well-specified, but their presence serves as strong signals for human developers to judge the report's specificity and repairability.

Based on this insight, we manually annotated the SWE-bench dataset for the presence or absence of key features (defined in Table 2) for the user-reported *Problem Statement* and the subsequent *Developer Hints* (if available). The detailed *Diagnostic Cue Annotation Schema* is provided in Appendix A. These annotations allow us to quantify the information richness of each issue and later correlate it with LLM or human judgments. The annotation process involved two of the authors collaboratively sampling a representative subset of SWE-bench issues guided by OpenAI's human annotation subset. The annotators first jointly reviewed a sample to develop consistent criteria for identifying each feature. They then independently labeled all issues, marking the existence of each diagnostic feature. Disagreements were resolved through discussion with a third author to ensure consistency and reliability. Across the 1,699 reports in SWE-bench-1699, we observed 129 disagreements (7.5%), with a Cohen's Kappa of 0.85, indicating strong inter-annotator agreement. Most disagreements stemmed from annotator fatigue when reading lengthy reports, leading to occasional missed or incorrect labels.

*3.4.2 Static and Dynamic Informativeness in Human-Labeled Data.* Because issue specification can be reflected both in the initial report and in later clarification during issue resolution, we separate static informativeness from dynamic incremental informativeness. This separation allows us to distinguish information already available to the agent before execution from information that becomes visible through developer follow-up.

To characterize how issue-report informativeness is reflected in human-labeled data and real debugging interactions, we analyze RQ1 from two complementary perspectives: (1) the diagnostic cues associated with human judgments of problem specification, and (2) the additional information introduced through developer follow-up during issue resolution.

**Static problem specification informativeness.** We first examine which diagnostic cues in the initial issue report are associated with human judgments of whether the report is well specified. Specifically, we compare the distribution of key diagnostic features across issue subsets with different levels of *Problem Specification*. This analysis reveals which cues are more commonly present in reports judged by humans to be sufficiently informative. We further apply Chi-square tests to assess whether differences in cue presence are statistically significant.

**Dynamic incremental informativeness.** We then examine how missing information is supplemented during real issue resolution. We compare diagnostic cues in the initial *Problem Statement* with those introduced later in *Developer Hints*, and visualize these transitions using Sankey diagrams. In this setting, the absence of follow-up hints suggests that the initial report was sufficiently informative for diagnosis, whereas the presence of developer follow-up indicates residual underspecification in the original report. This analysis characterizes the dynamic information structure

of issue diagnosis by showing what types of information are added after the initial report and therefore contributed incrementally to issue resolution.

*3.4.3 Information Gain Analysis.* The cue-association analysis identifies individual cues, but it does not quantify how much uncertainty about the specification label is reduced by report-side and hint-side cues. We therefore use information gain to quantify their aggregate and incremental informativeness.

An information-gain view is adopted to quantify how much diagnostic information is conveyed by the initial *Problem Statement* $R$ (static) and subsequent *Developer Hints* (dynamic). $Q \in \{0, 1\}$ denotes the human label indicating whether the *Problem Statement* is underspecified and requires additional information beyond the information provided in $R$. From $R$, a binary diagnostic-cue vector $X_R \in \{0, 1\}^m$ is extracted, as described in §2, where the $i$-th entry indicates whether cue $c_i$ (from a fixed diagnostic cue taxonomy; see §3.4.1) is present in $R$. For the same issue, a binary cue vector $X_H \in \{0, 1\}^m$ is extracted from the corresponding *Developer Hints* using the same taxonomy. The joint diagnostic feature set $(X_R, X_H)$ is formed by concatenating the two cue vectors.

*Static information gain.* Recall that $Q \in \{0, 1\}$ denotes the human label indicating whether the *Problem Statement* is underspecified and requires additional information beyond the information provided in $R$. The informativeness of *Problem Statement* cues about $Q$ is measured as

$$IG_Q(R) = H(Q) - H(Q \mid X_R), \tag{1}$$

where $H(\cdot)$ denotes Shannon entropy. $H(Q)$ is the entropy of the empirical specification label $Q$ distribution, and $H(Q \mid X_R)$ is the conditional uncertainty after observing diagnostic cues. $IG_Q(R)$ therefore measures the diagnostic informativeness of the *Problem Statement* relative to the empirical specification label prior.

*Dynamic information gain.* The additional informativeness of *Developer Hints* beyond the report is measured as

$$IG_Q(H \mid R) = H(Q \mid X_R) - H(Q \mid X_R, X_H). \tag{2}$$

$IG_Q(H \mid R)$ quantifies the *incremental* uncertainty reduction attributable to cues in *Developer Hints* after conditioning on the same *Problem Statement* cues.

*Estimation with logistic regression.* Each issue has a fixed observed label $Q$, while this label is modeled as a random variable over issues. We operationalize conditional entropy using out-of-fold predictive log loss from logistic regression:

$$\widehat{H}(Q \mid X) = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(q_i \mid x_i), \tag{3}$$

where $p_\theta(q_i \mid x_i)$ is the held-out predicted probability assigned by the model. $\widehat{H}(Q)$ is estimated using an intercept-only model, corresponding to the empirical label prior. Substituting these estimates into Eqs. 1 and 2 yields $\widehat{IG}_Q(R)$ and $\widehat{IG}_Q(H \mid R)$. Logistic regression was selected because it provides a transparent estimator, is well suited to sparse binary cues, and reduces the risk that estimated information gain is driven by model expressiveness rather than the diagnostic signals themselves.

*Single-cue information analysis.* For each binary cue $x_i \in \{0, 1\}$, we report its **cue frequency**,

$$\text{Freq}(x_i) = \frac{1}{N} \sum_{j=1}^{N} \mathbf{1}[x_i^{(j)} = 1], \tag{4}$$

which is the fraction of issues in which cue $x_i$ is present. We also compute the mutual information between each cue and the specification label:

$$MI(Q; x_i) = H(Q) - H(Q \mid x_i). \tag{5}$$

For this single-cue analysis, $MI(Q; x_i)$ is estimated directly from the empirical joint distribution of $Q$ and $x_i$. Higher values indicate that the cue alone provides more information about the specification label $Q$.

*3.4.4 Human and LLM Judgment Alignment.* After establishing which cues are informative under human labels, we test whether LLM judgments reflect the same information by comparing explicit ratings with human ratings and by checking whether human-valued cues appear in implicit uncertainty signals.

To assess whether LLM judgments track human judgments under the same issue-report inputs, we consider two forms of judgment signal: explicit ratings elicited directly from the LLM models, and implicit uncertainty estimated from response stability under semantically equivalent inputs. In this context, response stability means that the model gives consistent judgments across meaning-preserving paraphrases of the same issue report; large variation across such paraphrases indicates higher implicit uncertainty.

**Explicit Judgment.** Using LLMs as judges has become a common practice in recent research, where LLMs are leveraged to approximate human judgment. Following this approach, we prompt LLMs to directly assess the annotated *SWE-bench-1699* dataset to examine their explicit judgments, specifically in terms of *Problem Specification*, *Task Difficulty*, and Confidence Level. To ensure alignment, we reuse the human annotation schema from OpenAI [36] as the instruction prompt for the LLMs. The full template is provided in Appendix B. To analyze agreement and deviation, we use histograms to compare the ratings provided by LLMs and human annotators. To ensure robustness, we repeated randomized-order prompting for the LLM Judge three times to mitigate potential positional bias in LLM judgments.

**Implicit Uncertainty Signal.** Explicit pre-execution self-judgments may not fully reflect how stable or uncertain a model's judgment is. We therefore complement them with a stability-based uncertainty proxy derived from semantic consistency [29, 31]. This method quantifies uncertainty in black-box LLMs based on their response variability to semantically equivalent inputs, where higher uncertainty corresponds to lower internal certainty. Specifically, for each *Problem Statement*, we generate $M = 5$ meaning-preserving paraphrases and compute *Semantic Eccentricity*, where the paraphrases are alternative surface forms of the same original *Problem Statement*, rather than multiple distinct issue statements. *Semantic Eccentricity* quantifies the average deviation of each output from the overall semantic consensus (i.e., the mean representation of all paraphrases). A higher eccentricity score indicates greater uncertainty and lower internal certainty, whereas a lower score reflects a more stable and confident understanding. This metric enables us to objectively evaluate the model's implicit judgment for each *Problem Statement*.

Based on semantic agreement scores, we categorized issues into high uncertainty (Eccentricity $> 0.5$) and low uncertainty ($\leq 0.5$) groups. We then analyzed the distribution of annotated diagnostic features across these groups, and conducted a Chi-square test to assess the statistical significance of differences in feature presence.

*3.4.5 Human and LLM Judgment Alignment Analysis.* We next quantitatively evaluate whether LLM judgments align with human judgments under the same issue-report inputs. We focus on two dimensions: *Problem Specification* and *Task Difficulty*. Let $\hat{s}$ and $\hat{d}$ denote the scores predicted by the model, and let $S, D \in \{0, 1, 2, 3\}$ denote the corresponding human ratings for *Problem Specification* and the ordinally encoded *Task Difficulty*, respectively, as defined in §3.1. We measure ordinal

Table 2. Diagnostic Features extracted from problem statements and developer hints.

| Problem Statement | Developer Hints | Description |
|---|---|---|
| `r1-steps` | `h1-steps` | Reproduction steps/test cases |
| `r2-stack` | `h2-stack` | Stack trace/error message |
| `r3-code` | `h3-code` | Code snippet |
| `r4-media` | `h4-media` | Images |
| `r5-docs` | — | Docs/links |

agreement using Spearman's rank correlation:

$$\rho_S = \text{Spearman}(\hat{s}, S), \qquad \rho_D = \text{Spearman}(\hat{d}, D). \tag{6}$$

Spearman's $\rho$ ranges from $-1$ to 1, where larger positive values indicate stronger ordinal agreement between model predictions and human ratings, $\rho = 0$ indicates no monotonic association, and negative values indicate ordinal disagreement between model and human labels. The $p$-value tests the null hypothesis of zero rank correlation.

### 3.5 RQ2: Pre-execution Confidence–Competence Alignment

RQ2 examines whether an LLM's pre-execution self-assessment is aligned with its downstream repair outcome under controlled cue perturbations. In this paper, **pre-execution** refers to judgments made before the model generates or executes a candidate fix. We use **pre-execution self-assessment** to refer to the model's two raw judgments, namely *Problem Specification* quality and *Task Difficulty*. We use **pre-execution confidence** to refer to the operational confidence score derived from these two self-assessment ratings. **Competence** refers to realized repair success, operationalized by the official SWE-bench evaluation outcome. **Alignment** refers to the extent to which changes in **pre-execution self-assessment** track corresponding changes in repair success.

Building on RQ1, which analyzes human and LLM judgments under matched information conditions, RQ2 investigates whether pre-execution self-assessment aligns with real repair performance under diagnostic-cue perturbations. Because our experimental lever is cue perturbation, we evaluate alignment at the change level: if a perturbation makes repair less likely to succeed, a calibrated pre-execution self-assessment should shift toward lower perceived specification quality or higher perceived difficulty.

We use issues from SWE-bench Verified that contain both reproduction steps (`r1-steps`) and stack traces (`r2-stack`). For each original issue, we define three target cue sets: (1) reproduction steps only, (2) stack traces only, and (3) both reproduction steps and stack traces. We then apply two intervention types to each target cue set: *information removal* and *information rephrasing*. This design yields six controlled variants per issue.

- **Information Removal**: removes the selected target cue(s) from the original issue. For example, under the "both cues" condition, both reproduction steps and stack traces are removed.
- **Information Rephrasing**: rewrites the selected target cue(s) while preserving its semantic content. For example, under the "both cues" condition, both reproduction steps and stack traces are rephrased rather than removed.

In our experiments, rephrasing is generated using an LLM-based paraphrasing procedure for each of the three target cue conditions: reproduction steps only, stack traces only, and both cues together. We use a dedicated prompt designed to preserve the underlying semantic content while altering only the surface wording. The prompt is: "You are a professional software engineer who

is responsible for analyzing software problems. Please strictly follow the classification system provided for analysis."

Together, these manipulations produce six systematic variants per issue (three cue conditions × two intervention types). Figure 2 shows an example of stack trace only rephrasing and removal. Using these controlled variants, we examine how the absence or rephrasing of human-valued cues affects repair success and whether changes in pre-execution self-assessment track the resulting changes in repair outcome.

*3.5.1 Change-based Alignment Analysis.* We use the same pre-execution self-assessment ratings as defined in RQ1, namely *Problem Specification* ($s_i$) and *Task Difficulty* ($d_i$), together with the binary repair outcome $G_i \in \{0, 1\}$ from the official SWE-bench evaluation, where $G_i = 1$ indicates a successful fix. Because RQ2 focuses on intervention-induced changes, we operationalize alignment relative to the original problem statement. For each issue $i$ and intervention variant $v$, let $(s_{i0}, d_{i0}, G_{i0})$ denote the ratings and repair outcome for the original input, and $(s_{iv}, d_{iv}, G_{iv})$ those for the modified variant. We define

$$\Delta s_{iv} = s_{iv} - s_{i0}, \qquad \Delta d_{iv} = d_{iv} - d_{i0}, \qquad \Delta G_{iv} = G_{iv} - G_{i0}. \tag{7}$$

Positive $\Delta s_{iv}$ and $\Delta d_{iv}$ indicate that the modified issue is judged to be less well specified and more difficult, respectively, while $\Delta G_{iv} < 0$ indicates degraded repair performance.

Change-based alignment is then measured by Spearman's rank correlation between self-assessment shifts and performance shifts:

$$\rho_S = \text{Spearman}(\Delta s, -\Delta G), \qquad \rho_D = \text{Spearman}(\Delta d, -\Delta G). \tag{8}$$

Larger positive values indicate stronger agreement between self-assessment change and performance degradation.

### 3.6 RQ3: Behavioral Response to Input Perturbations

RQ1 and RQ2 focus on diagnostic cues, pre-execution self-assessments, and downstream repair outcomes. However, outcome-level metrics alone cannot explain how agents adjust their execution process when these cues are removed or rephrased. RQ3 therefore examines **behavioral response** in repair trajectories under perturbed inputs. In this paper, a **repair trajectory** refers to the observable sequence of high-level operations performed by an agent during one issue-resolution attempt, from initial localization and code inspection to modification, testing, recovery, and termination.

We use the term behavioral response to refer to observable trajectory-level changes induced by input perturbations, rather than learning, memory, or self-adjustment across runs.

*3.6.1 Agent Operation Tree.* To analyze such trajectories beyond final outcomes, we represent each agent run as a hierarchical **Agent Operation Tree**. This structure captures the agent's repair trajectory, including both the sequence of operations and higher-level behavioral patterns such as debugging loops (modification → test → modification) and rollback behavior. We classify agent operations into six semantic categories:

- **File Localization:** Identifying relevant files required for the task.
- **Thinking:** High-level reasoning or planning operations.
- **Code Analysis:** Inspecting code structure, content, or dependencies.
- **Code Modification:** Editing or repairing source code.
- **Test Execution:** Running tests or verification commands.
- **Task Completion:** Explicit task termination (e.g., successful repair or agent exit).

Each node in the tree represents a high-level operation derived from low-level tool calls. Nodes are further enriched with execution metadata, such as success/failure status, error details, and

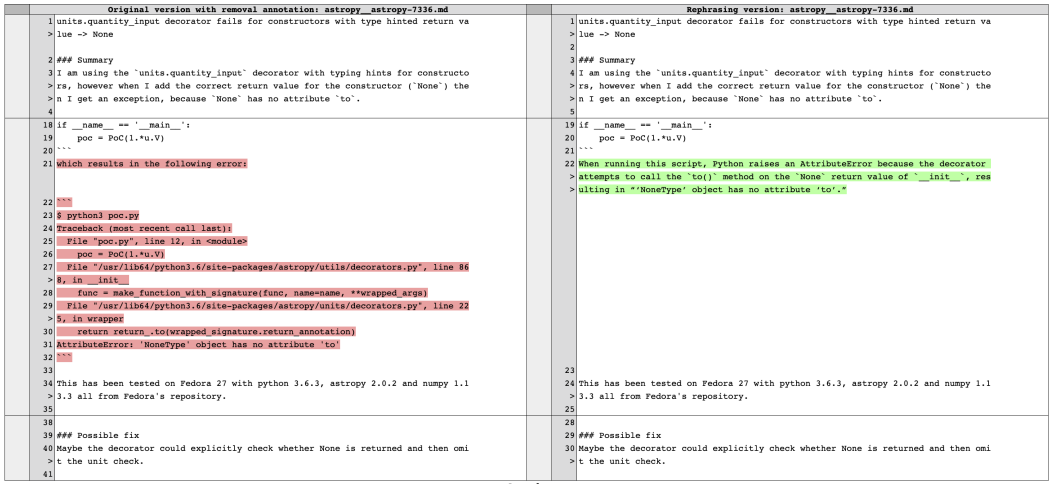


Fig. 2. Example of stack trace only removal (left, red) and rephrasing (right, green) for an issue report.

tool-specific context. The tree structure is built according to logical parent-child relationships based on operation types and file-level dependencies, as illustrated in Figure 3. We define a **rollback** as a transition from a test execution or code modification on one file to a subsequent code analysis on a different file. Irrelevant traces (e.g., quit/exit commands or operations without clear file context) are excluded.

*3.6.2 Trajectory Feature Extraction and Attribution Analysis.* While the trajectory tree provides a qualitative, visual map of the agent's workflow, a quantitative approach is necessary for systematic and scalable analysis. To achieve this, we summarize each trajectory tree as a fixed-dimensional behavioral feature representation (see Table 3). The representation has the same set of features for every trajectory, but the feature values capture how the trajectory unfolds, including execution effort, path length, error handling, and testing behavior. Thus, when we later describe a trajectory as becoming longer, we refer to increases in trajectory-level features such as *total rounds*, *total executions*, and *longest path length*, not to a change in the dimensionality of the feature vector.

These features capture multiple dimensions of scaffold-mediated agent behavior, including overall efficiency, error handling, modification behavior, and testing behavior. Because all trajectories are generated within the OpenHands scaffold, we interpret these features as observable behaviors of the complete agent system rather than direct evidence of model-internal strategic adaptation. This distinction is important for features such as rollback count, testing frequency, and execution length, which may reflect both model decisions and scaffold-level control logic during execution.

For the perturbation analysis, we first compute trajectory-feature vectors for the original unaltered issues that contain both diagnostic cues. These original runs form the baseline condition. We then compute the same feature vectors for each of the six controlled variants introduced in §3.5: removal or rephrasing of reproduction steps, stack traces, or both. For each feature and each variant, we compare the distribution of feature values against the corresponding baseline distribution and report the relative percentage change in the mean feature value. Formally, for feature $f$ and variant $v$, the reported change is computed as

Table 3. Statistical features extracted from each agent trajectory for attribution analysis.

| Feature Name | Definition |
|---|---|
| *Overall Efficiency* | |
| `total_cost` | Total monetary cost of the trajectory |
| `total_rounds` | Number of agent-environment interaction rounds |
| `total_executions` | Total operations executed |
| `longest_path_length` | Max continuous operations of the same file |
| *Error Handling* | |
| `failed_executions` | Number of operations resulting in error execution |
| `failure_exe_repair_count` | Failed operations followed by successful retry |
| `rollback_count` | Number of times the agent reverts to code analysis on a different file |
| *Modification & Testing Behavior* | |
| `modification_attempts` | Number of code modification attempts |
| `test_execution_count` | Total number of test executions |
| `last_test_success` | Whether the final executed test was successful |
| `test_before_code_mod` | Tests executed before any code modification |
| `test_only_after_code_mod` | Tests executed after code modification |
| `test_between_code_mod` | Tests between code modifications |

$$\Delta_f^{(v)} = \frac{\overline{f}^{(v)} - \overline{f}^{(\text{base})}}{\overline{f}^{(\text{base})}} \times 100\%. \tag{9}$$

Positive values indicate that the feature increases under the perturbation, while negative values indicate a decrease. To assess whether the observed feature shifts are statistically reliable, we use the Mann–Whitney U test to compare the baseline and variant distributions for each feature, since the trajectory features are often non-normal counts or skewed continuous values. We additionally report Cohen's $d$ as an effect-size estimate to indicate the magnitude of the observed difference.

*3.6.3 Logistic Regression Analysis.* To further assess whether the extracted trajectory features capture behavior associated with repair success, we train a logistic regression model to predict issue-resolution outcome from these features. This analysis is not intended to establish a causal relationship between individual trajectory features and success. Instead, it provides an interpretable association analysis that helps identify which observable operation patterns tend to co-occur with successful or failed repairs. By examining coefficient signs and magnitudes, we identify which forms of behavioral response are associated with successful repair, which are associated with failure, and which appear less informative. This complements the descriptive trajectory analysis by linking trajectory changes under perturbation to downstream repair outcomes in a transparent and reproducible manner.

### 3.7 RQ4: Toward Self-Judgment-Based Correction

We frame RQ4 as a post-execution monitoring problem. After a repair attempt has produced a candidate patch and trajectory, we examine whether post-execution self-judgment and observable trajectory evidence can detect likely failed attempts and support subsequent corrective decision-making as complementary validation signals.

*3.7.1 Post-execution Self-Judgment (PSJ).* Building on the behavioral insights from RQ3, we introduce an LLM-based *Post-execution Self-Judgment* module that re-evaluates the agent's pre-execution

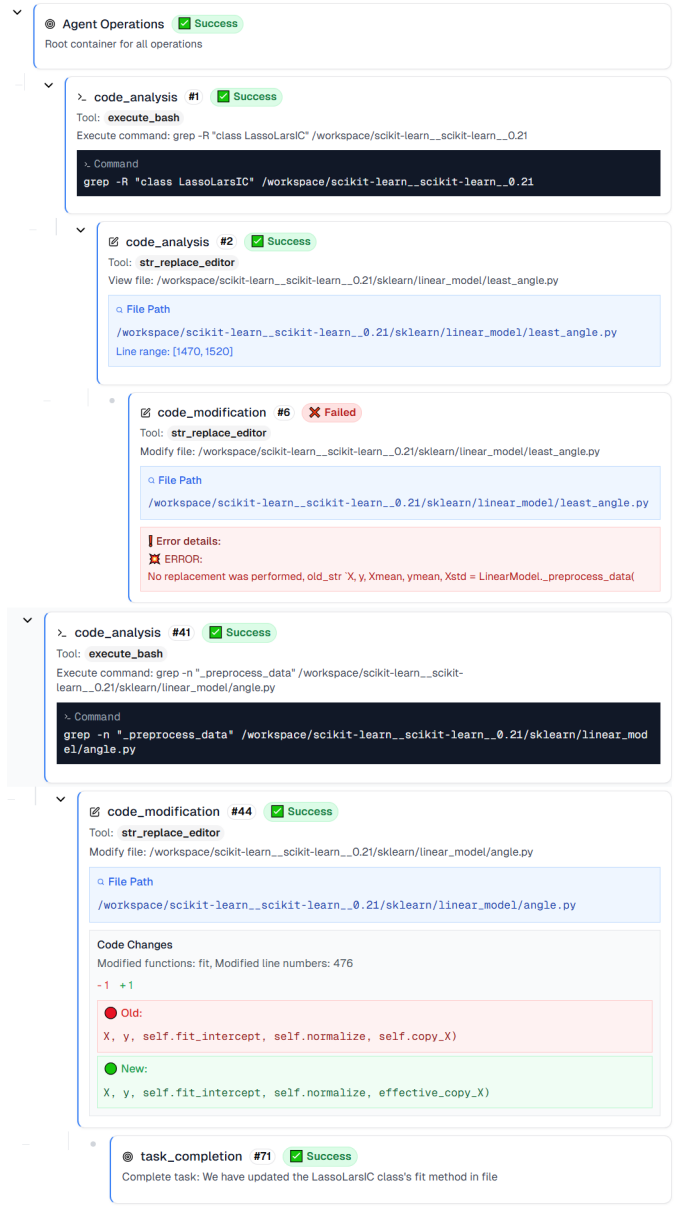


Fig. 3. Example Operation Tree visualizing an agent's trajectory, including logical operation flow, success/-failure steps, and rollbacks.

assessments after observing the completed repair process. The PSJ module receives: (1) the agent's original specification and difficulty ratings, together with their justifications (§3.4.4); (2) the Agent Operation Tree summarizing repair steps (§3.6.1); and (3) extracted trajectory features (§3.6.2).

The PSJ module outputs: (1) revised specification and difficulty ratings; (2) a bug-fixing score as an auxiliary self-evaluation signal [1] (0 = no or incorrect fix, 1 = partial fix, 2 = complete fix); (3) a

100-word summary of the execution trajectory; (4) a binary flag indicating any critical trajectory issues; and (5) a short description of potential improvements. These outputs allow us to evaluate whether post-execution self-judgment changes after execution in ways that are informative about the realized repair process and outcome.

*3.7.2 Logistic Models with Behavioral and PSJ Features.* We further integrate the trajectory-level behavioral features derived from RQ3 with the features produced by PSJ to test whether post-execution self-judgment provides complementary information beyond behavior alone. Following §3.6.3, we fit logistic regression models using behavioral features only, PSJ features only, and their combination. This design isolates the incremental value of PSJ signals for failure detection in completed repair attempts, rather than optimizing absolute predictive performance.

*3.7.3 Failure Detection Analysis.* Failure detection is evaluated against the external SWE-bench outcome $G_i \in \{0, 1\}$, where $G_i = 1$ indicates a successful fix and $G_i = 0$ indicates failure. Given a scalar failure score $\hat{p}_i$ produced by a method (either the judge module's direct binary judgment or a learned model), predictive performance is summarized by the area under the ROC curve:

$$\mathrm{AUC} = \mathrm{AUC}(\hat{p}, G), \tag{10}$$

where larger AUC indicates stronger discriminability between successful and failed runs.

### 3.8 Implementation

To power this framework and provide necessary comparisons, we selected a diverse set of LLMs. We experiment with **Claude Sonnet 4** for its superior proprietary performance, **GPT o4-mini** to assess cross-model robustness, and **GPT-4.1** as a representative non-thinking model. This configuration enables a robust evaluation of agent scaffolding and model variation in automated software repair.

We adopt **OpenHands** [45] as the experimental agent scaffolding framework, reflecting its state-of-the-art performance on SWE-bench, as well as its wide adoption and investigation in recent research [12, 37, 42, 43]. OpenHands enables LLMs to move beyond static code generation by supporting file editing, command execution, and dynamic analysis of execution outputs in a secure, sandboxed environment. This interactive workflow allows the agent to iteratively detect errors and refine solutions based on observed outcomes. This work is not intended to optimize leaderboard agent performance; rather, it conducts a principled investigation into the reasoning behaviors and cognitive dynamics of LLM-based agents during automated software repair.

## 4 Results

### 4.1 RQ1: Problem Specification Informativeness and Judgment Alignment

RQ1 evaluates problem specification informativeness and judgment alignment from two perspectives. First, from the human side, we characterize issue-report informativeness in the human-labeled data through static cue association, dynamic developer follow-up, and information-gain analyses, which together explain why static, dynamic, and aggregate informativeness are all needed to assess problem specification. Second, from the LLM side, we test whether LLMs' explicit judgments and implicit uncertainty signals reflect the same specification-relevant information.

*4.1.1* ***Static Problem Specification Informativeness from a Human Perspective.*** To examine whether human judgments of problem statement specificity are associated with particular diagnostic cues, we analyzed annotated diagnostic features on the SWE-bench-1699 and Verified datasets (Figure 4). In the SWE-bench-1699 analysis, `r1-steps` shows a statistically significant but small association with human judgments of specification quality ($\chi^2 = 17.16$, $p < 0.001$, Cramér's $V = 0.087$). Stack traces (`r2-stack`) show a weaker association ($\chi^2 = 4.57$, $p = 0.033$, $V = 0.045$),

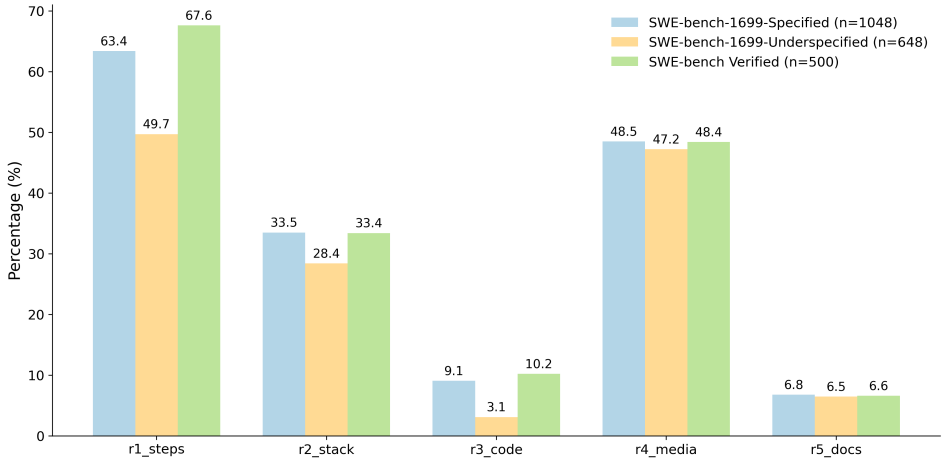


Fig. 4. Distribution of annotated diagnostic features across the SWE-bench datasets. The y-axis reports the percentage of problem statements within each dataset subset that contain each diagnostic feature: r1 = reproduction steps, r2 = stack trace, r3 = code snippets, r4 = images, and r5 = docs/links. The compared subsets are SWE-bench-1699–Specified (n = 1,048), SWE-bench-1699–Underspecified (n = 648), and SWE-bench–Verified (n = 500).

again with a small effect size. Code snippets (`r3-code`) are also associated with specification quality ($\chi^2 = 13.45$, $p < 0.001$, $V = 0.077$), though the effect size remains small. We exclude this feature from subsequent experiments because it may cause answer leakage [2] by directly providing a solution rather than reflecting an LLM's own assessment ability. Images (`r4-media`) and documentation/links (`r5-docs`) show no statistically significant association and are omitted from further analyses. While these features may be relevant for multimodal or retrieval-augmented settings, they are beyond the scope of this study.

Practically, these small effect sizes mean that reproduction steps and stack traces are useful diagnostic cues, but neither cue alone is sufficient to explain human problem-specification judgments. The relationship between issue specification and diagnostic cues is therefore more complex than any single feature alone can capture, motivating the information-gain analysis in §3.4.3.

**Observation RQ1-1:** Reproduction steps are the strongest single report-side cue associated with human problem-specification ratings, but the association is small: `r1-steps` has $\chi^2 = 17.16$, $p < 0.001$, and Cramér's $V = 0.087$. Stack traces (`r2-stack`) show a weaker small association ($\chi^2 = 4.57$, $p = 0.033$, $V = 0.045$). Thus, individual diagnostic cues are useful signals, but none is sufficient by itself to explain human specification judgments.

*4.1.2* ***Dynamic Incremental Informativeness from Developer Follow-up.*** To move beyond a static analysis of *Problem Statements* and better capture real-world debugging interactions, we additionally annotated key diagnostic features in *Developer Hints*, which refer to developer/maintainer follow-up comments as defined in §3.4.1. The Sankey diagram in Figure 5 visualizes how reproduction steps and stack traces are introduced, preserved, or supplemented across the discussion, tracking these features from the initial *Problem Statement* (`r1-steps` and `r2-stack`) to *Developer Hints* (`h1-steps` and `h2-stack`). All feature labels are defined in Table 2. This analysis reveals several recurring communication patterns.

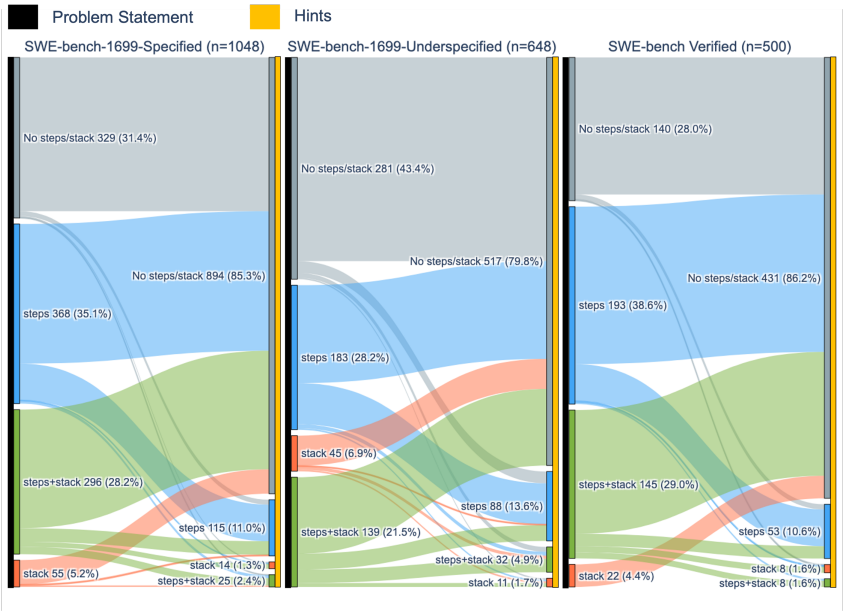


Fig. 5. Sankey diagram illustrating how manually annotated diagnostic cues of *reproduction steps* and *stack traces* flow from issue reporters to repository developers during real debugging interactions in SWE-bench issue threads. All features were manually annotated by our annotators. The left black blocks ("Problem Statement") denote cues extracted from the reporter's initial issue description, and the right yellow blocks ("Hints") denote cues extracted from subsequent developer follow-up comments. The same manual annotation schema is applied to both sides. Flow width indicates the number of issues following each transition pattern.

**No Initial Diagnostic Information:** Reports lacking both reproduction steps and stack traces are less frequent in the *Specified* set than in the *Underspecified* set (31.4% vs. 43.4%). These issues interestingly receive minimal follow-up, suggesting they often correspond to simple or self-contained bugs that do not require additional clarification.

**Stack Traces Only:** Reports containing only stack traces comprise a small fraction of issues (5.2% in the *Specified* set and 6.9% in the *Underspecified* set). Such reports rarely trigger follow-up, indicating that stack traces alone can sometimes provide sufficient grounding for diagnosis, especially for execution-related failures.

**Reproduction Steps Only:** In the *Specified* set, 35.1% of reports (368/1048) contain only reproduction steps, and among these, 23% (85/368) involve follow-up developer interactions such as requests for additional reproduction details or stack traces. In the *Underspecified* set, 28.2% of reports (183/648) include only reproduction steps, with a higher follow-up rate of 32% (59/183). This difference suggests that when reproduction steps are clearly specified, developers are more likely to resolve issues in a single attempt, whereas underspecified reports often require additional clarification or verification.

**Both Steps and Stacks Provided:** Reports including both reproduction steps and stack traces constitute the most complete initial diagnosis. This category is more prevalent in the *Specified* set (28.2%), representing a 6.7% increase over the *Underspecified* set, and it consistently requires the least follow-up. These cases span a broad range of issue types, underscoring their importance for both research and practice.

**Observation RQ1-2:** Developer follow-up shows that specification is partly interactional. Reports without reproduction steps or stack traces are more common in the *Underspecified* set than in the *Specified* set (43.4% vs. 31.4%), and reproduction-steps-only reports receive follow-up more often when *Underspecified* than when *Specified* (32% vs. 23%). Reports containing both reproduction steps and stack traces are more common in the *Specified* set (28.2%, 6.7 percentage points higher than in the *Underspecified* set), suggesting that combined diagnostic grounding is associated with fewer clarification needs.

*4.1.3* ***Information Gain Analysis.*** We quantify how much information about the operational specification label $Q$ is conveyed by the initial *Problem Statement* and how much additional information is contributed by subsequent *Developer Hints*. This analysis is conducted on the annotated SWE-bench-1699 subset after excluding instances without valid operational specification labels. The resulting valid subset contains 1,696 instances, of which 1,048 instances (61.8%) are labeled as specified ($Q = 0$), while 648 instances (38.2%) are labeled as underspecified ($Q = 1$). In this analysis, $H(Q)$ denotes the prior uncertainty of the binary specification label before observing any diagnostic cue. Following our cross-validated log-loss estimation procedure, the intercept-only baseline gives an estimated prior uncertainty of $\widehat{H}(Q) = 0.688$ nats, which is close to the maximum entropy of a balanced binary label distribution.

Using five-fold cross-validated logistic regression to estimate conditional entropy, we obtain $\widehat{H}(Q \mid X_R) = 0.666$ and $\widehat{H}(Q \mid X_R, X_H) = 0.376$. The initial *Problem Statement* yields only limited information gain, $\widehat{IG}_Q(R) = 0.0217$ (3.15% of $\widehat{H}(Q)$), whereas adding *Developer Hints* contributes substantially more, $\widehat{IG}_Q(H \mid R) = 0.2900$ (42.15% of $\widehat{H}(Q)$). The contrast between 3.15% uncertainty reduction from the initial report and 42.15% after adding developer hints gives a concrete magnitude to the static-versus-dynamic distinction: whether an issue is operationally specified is only weakly signaled by the static initial report and becomes much clearer through subsequent developer interaction.

Table 4 further reports per-feature mutual information $MI(Q; x_i)$. Larger $MI(Q; x_i)$ values indicate that observing cue $x_i$ reduces more uncertainty about whether the issue is under-specified. The percentage column in Table 4 reports this reduction relative to the estimated prior uncertainty, i.e., $MI(Q; x_i)/\widehat{H}(Q) \times 100\%$. Among *Problem Statement* cues, `r1_steps` is the strongest individual signal, but its contribution remains small ($MI = 0.0182$, $MI/\widehat{H}(Q) = 2.64\%$), whereas `r2_stack` contributes a smaller effect and the remaining report-side cues are negligible. Among *Developer Hints*, `h1_steps` is the strongest non-leaking cue ($MI = 0.1400$, $MI/\widehat{H}(Q) = 20.34\%$), followed by `h2_stack` ($MI = 0.0399$, $MI/\widehat{H}(Q) = 5.80\%$). We exclude `h3_code` from substantive interpretation because it may cause answer leakage.

**Observation RQ1-3:** Information-gain analysis quantifies the gap between static and dynamic information. Initial *Problem Statement* cues reduce uncertainty about the specification label by only 0.0217 nats, or 3.15% of $\widehat{H}(Q)$, whereas adding *Developer Hints* reduces uncertainty by 0.2900 nats, or 42.15% of $\widehat{H}(Q)$. At the single-cue level, `h1_steps` contributes 20.34% and `h2_stack` contributes 5.80%, compared with 2.64% for `r1_steps` and 0.63% for `r2_stack`. This suggests that later developer interaction provides substantially more specification-relevant information than the initial issue report alone.

*4.1.4* ***Human–LLM Judgment Alignment: Explicit Assessment.*** We next examine whether LLM judgments align with human ratings under the same issue-report inputs. Figure 6 shows

Table 4. Per-feature cue frequency and mutual information $MI(Q;x_i)$ with the operational specification label $Q$. $\widehat{H}(Q) = 0.688$ nats denotes the estimated prior uncertainty of the binary specification label, and the last column reports the relative uncertainty reduction, computed as $MI(Q;x_i)/\widehat{H}(Q) \times 100\%$. We exclude h3_code from substantive discussion due to answer leakage.

| Diagnostic Feature | Cue Frequency | $MI(Q;x_i)$ | $MI(Q;x_i)/\widehat{H}(Q)$ (%) |
|---|---|---|---|
| h1_steps | 15.3% | 0.1400 | 20.34 |
| h2_stack | 4.8% | 0.0399 | 5.80 |
| r1_steps | 58.7% | 0.0182 | 2.64 |
| h4_media | 2.0% | 0.0160 | 2.32 |
| r2_stack | 31.2% | 0.0043 | 0.63 |
| r3_code | 6.9% | 0.0040 | 0.59 |
| r5_docs | 7.3% | 0.0030 | 0.43 |
| r4_media | 48.5% | 0.0002 | 0.02 |
| h3_code (excluded) | 19.0% | 0.1796 | 26.10 |

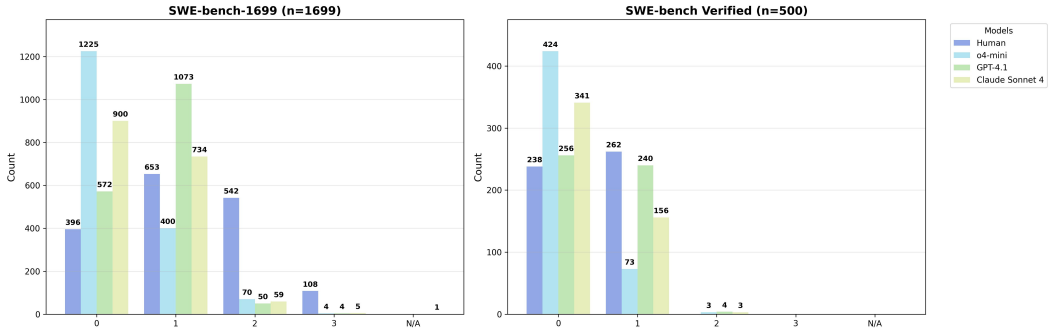


Fig. 6. Comparison between OpenAI human annotators and LLM judges (*o4-mini*, *GPT-4.1*, *Claude Sonnet 4*) on issue **Problem-Specification** assessments across the SWE-bench-1699 and SWE-bench-Verified datasets. The x-axis shows the four problem-specification score levels, where scores 0–1 correspond to well-specified or mostly specified issues, and scores 2–3 correspond to underspecified issues. The reported counts refer to the number of judgments assigned to each score level.

that the OpenAI annotators assign problem-specification ratings across the full range of the label scale, reflecting substantial variation in the quality and completeness of real-world issue reports. In contrast, all evaluated LLMs exhibit pronounced bias: both o4-mini and Claude Sonnet 4 rate the overwhelming majority of issues as "very clear" (Level 0), while GPT-4.1, though slightly less extreme, still concentrates ratings at "clear" (Level 1) (see Figure 6). This pattern persists even on the pre-filtered SWE-bench Verified set. Whereas human ratings remain balanced between "very clear" and "clear" (levels 0–1), LLMs disproportionately assign the highest clarity score. For example, o4-mini rates 424 issues as "very clear", 78.2% more than the human Level 0 count, and Claude Sonnet 4 rates 341 issues as "very clear", 43.3% more than the human count. This further demonstrates insufficient sensitivity to nuanced differences in problem-specification quality.

Figure 7 further shows that LLMs tend to assign higher task-difficulty ratings than human annotators. Rather than separating poor specification from intrinsic task difficulty, the models often


Fig. 7. Comparison between OpenAI human annotators and LLM judges (*o4-mini*, *GPT-4.1*, *Claude Sonnet 4*) on issue-**Task Difficulty** across the SWE-bench-1699 and SWE-bench-Verified datasets. The reported counts refer to the number of judgments assigned to each score level.

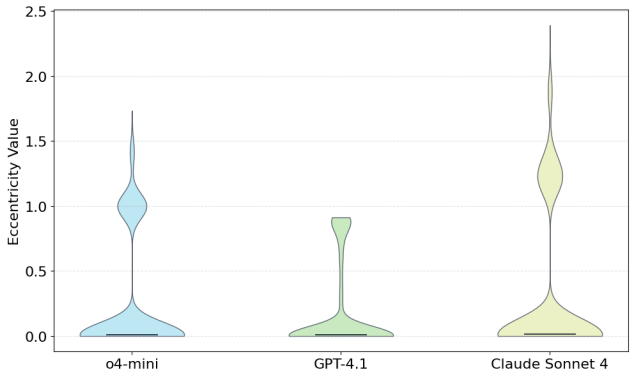


Fig. 8. Distribution of LLM uncertainty measured by **eccentricity** values. Each violin plot represents the overall spread of eccentricity across issue samples for a given model (*o4-mini*, *GPT-4.1*, and *Claude Sonnet 4*). Diagnostic-cue-specific eccentricity distributions are reported in Appendix C.

appear to map lack of clarity onto greater expected difficulty. For example, in the SWE-bench-1699 dataset, GPT-4.1 labels 622 issues as highly difficult, including 614 issues rated as 1–4 hours and 8 issues rated as more than 4 hours. This corresponds to 36.6% of the dataset, where "highly difficult" denotes issues expected to require more than one hour to resolve. In contrast, human annotators rate only 22.1% of issues as highly difficult. Moreover, while around 96% of issues are judged as "well-specified" (Levels 0–1) by the LLM judges, humans rate only 61.7% as such.

**Observation RQ1-4:** LLM judges rate issue reports as clearer than human annotators on the 0–3 *Problem Specification* scale. Across the evaluated LLM judges, around 96% of issues are rated as well-specified (levels 0–1), compared with 61.7% by human annotators. On SWE-bench Verified, o4-mini assigns 424 issues to Level 0, 78.2% higher than the human Level 0 count of 238, while Claude Sonnet 4 assigns 341 issues to Level 0, 43.3% higher than the human count. This indicates an optimistic shift in LLM specification ratings.

*4.1.5* ***Human–LLM Judgment Alignment: Implicit Uncertainty Signal.*** We further examine whether the diagnostic cues associated with human judgments are also reflected in an LLM's

Table 5. Spearman rank correlations between LLM judgments and human judgments on *Problem Specification* ($\rho_S$) and *Task Difficulty* ($\rho_D$). Values closer to 1 indicate stronger alignment with human ratings. All correlations are statistically significant (all $p < 10^{-4}$).

| | SWE-bench-1699 | | SWE-bench-Verified | |
|---|---|---|---|---|
| **Model** | $\rho_S$ | $\rho_D$ | $\rho_S$ | $\rho_D$ |
| Claude Sonnet 4 | 0.334 | 0.568 | 0.195 | 0.482 |
| GPT-4.1 | 0.389 | 0.590 | 0.231 | 0.492 |
| o4-mini | 0.360 | 0.561 | 0.174 | 0.494 |

implicit uncertainty signal. Figure 8 shows the overall distribution of uncertainty measured by semantic eccentricity across issue samples for *o4-mini*, *GPT-4.1*, and *Claude Sonnet 4*. To directly examine cue presence, Appendix C further splits the eccentricity distributions by whether each annotated diagnostic cue is absent or present. Statistical tests reveal no significant differences in cue presence between high- and low-uncertainty issues ($p > 0.05$ for all features), and effect sizes are negligible (Cramér's $V < 0.02$).

**Observation RQ1-5:** Human-valued diagnostic cues are not reliably reflected in the implicit uncertainty proxy. Cue-presence comparisons between high- and low-eccentricity groups are not statistically significant ($p > 0.05$ for all features), and effect sizes are negligible (Cramér's $V < 0.02$). Thus, this black-box uncertainty signal does not provide evidence that the cues humans associate with specification quality systematically make LLM judgments less stable.

*4.1.6* ***Human and LLM Judgment Alignment Analysis.*** Measured by Spearman's rank correlation, ordinal agreement between LLM judgments and human judgments is limited for both *Problem Specification* and *Task Difficulty* (Table 5). Since stronger ordinal agreement would be reflected by $\rho$ values closer to 1, we use *limited* to indicate that the observed correlations fall short of strong ordinal agreement: *Problem Specification* remains below 0.4 across all settings, while *Task Difficulty* remains below 0.6. On SWE-bench-1699, alignment is weak-to-moderate for *Problem Specification* ($\rho_S \in [0.334, 0.389]$) and moderate for *Task Difficulty* ($\rho_D \in [0.561, 0.590]$). On SWE-bench-Verified, the same pattern remains but becomes noticeably weaker: $\rho_S$ drops to 0.174–0.231, indicating low ordinal agreement, while $\rho_D$ remains somewhat higher but still stays below 0.5 (0.482–0.494), indicating only moderate agreement. Overall, these results reveal a persistent alignment gap: LLM judges are more aligned with human judgments of *Task Difficulty* than with human judgments of *Problem Specification*, consistent with Observation RQ1-4.

**Observation RQ1-6:** Rank-correlation analysis confirms that LLM-human agreement is consistently weaker for *Problem Specification* than for *Task Difficulty*. On SWE-bench-1699, specification agreement is weak-to-moderate ($\rho_S = 0.334$–$0.389$), while difficulty agreement is higher ($\rho_D = 0.561$–$0.590$). On SWE-bench-Verified, specification agreement drops to $\rho_S = 0.174$–$0.231$, while difficulty remains $\rho_D = 0.482$–$0.494$; all correlations are statistically significant ($p < 10^{-4}$). This means LLMs preserve human difficulty rankings better than human specification rankings, even though neither alignment is strong.

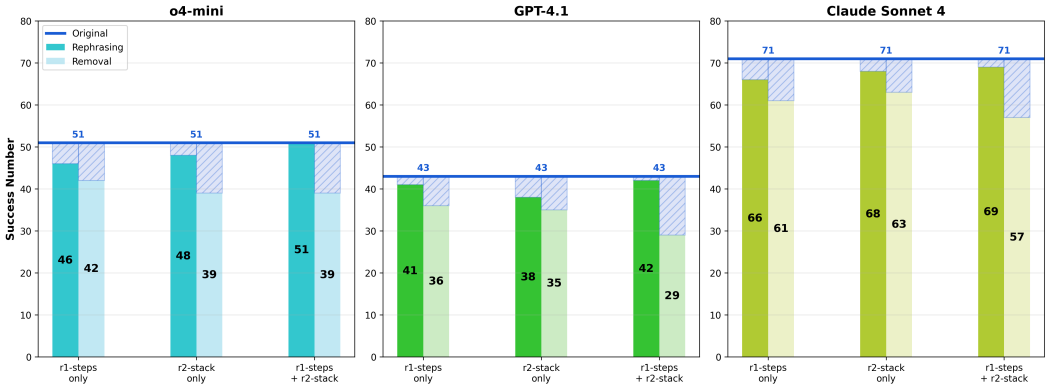


Fig. 9. Impact of diagnostic information removal and rephrasing on agent success rate for *o4-mini*, *GPT-4.1*, and *Claude Sonnet 4*. Bars show success counts under three feature configurations: `r1-steps`, `r2-stack`, and both combined, tested with original, rephrased, and removed information.

**Finding for RQ1:** Observations RQ1-1 to RQ1-3 show that human problem-specification judgments are grounded in diagnostic information, but that this information is distributed unevenly across the initial report and later interaction. Individual report-side cues have only small associations with human labels (e.g., `r1-steps` Cramér's $V = 0.087$ in Observation RQ1-1), while *Developer Hints* provide much larger incremental information than the initial *Problem Statement* alone (42.15% vs. 3.15% relative uncertainty reduction with respect to $\widehat{H}(Q)$ in Observation RQ1-3). Observations RQ1-4 to RQ1-6 show that LLM judges only partially align with this human-centered structure: they rate many more issues as well-specified (levels 0–1) than humans do (about 96% vs. 61.7%) in Observation RQ1-4, their implicit uncertainty signal does not reliably reflect human-valued diagnostic cues in Observation RQ1-5, and their ratings show comparatively higher agreement with human *Task Difficulty* rankings than with *Problem Specification* rankings in Observation RQ1-6. Overall, RQ1 shows that LLM judges tend to rate issue reports as clearer than humans do, while their uncertainty and ranking behavior do not reliably reflect human-valued specification cues.

## 4.2 RQ2: Pre-execution Confidence–Competence Alignment

Building on the findings of RQ1 and the setup described in §3.5, we identified a subset of 146 issues containing both `r1-steps` and `r2-stack`, which serves as the controlled testbed for evaluating whether changes in pre-execution self-assessment track changes in repair success. Following the ablation setup in §3.5, this analysis examines how two diagnostic cues, reproduction steps and stack traces, affect repair success and pre-execution self-assessment by comparing the original issues with six controlled variants, covering three cue conditions under removal and rephrasing interventions.

*4.2.1* ***Impact on Task Success Rate.*** Chi-square tests revealed that removing both `r1-steps` and `r2-stack` led to a significant decline in agent success rate ($\chi^2 = 9.71$, $p = 0.0018$). Specifically, GPT-4.1's baseline success count of 42 dropped to 29 (a 8.9 % decrease) when both cues were removed. Rephrasing these diagnostic cues also reduced performance, though the effect was smaller and not consistently significant (see Figure 9). This means the both-cue removal condition is the main setting where missing diagnostic content produces a practically visible performance loss.

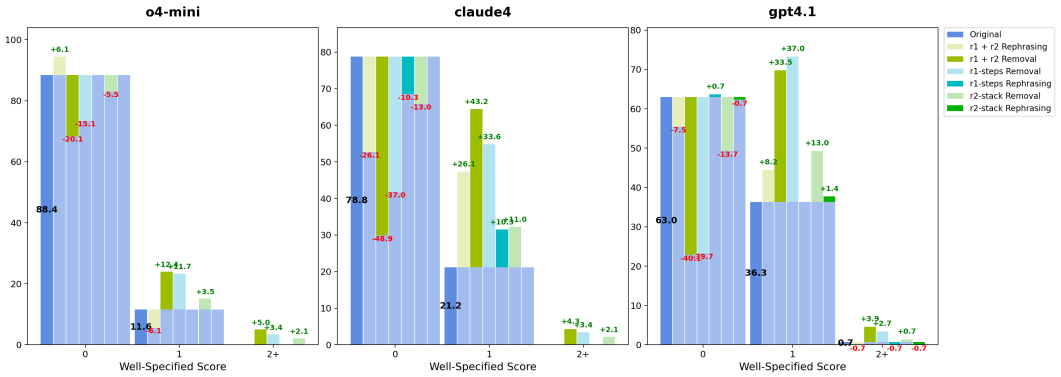


Fig. 10. Distribution of **Problem Specification** self-assessment scores across diagnostic information conditions. The y-axis refers to the percentage of judgments at each score level. Bars compare original, rephrased, and removed cue settings (`r1-steps`, `r2-stack`) for *o4-mini*, *Claude Sonnet 4*, and *GPT-4.1*.

In contrast, removing either `r1-steps` or `r2-stack` did not produce a significant performance decrease, particularly for Claude Sonnet 4, which showed the highest overall success rate. This pattern suggests a compensatory effect: the presence of either a natural-language description or one diagnostic cue can partially mitigate information loss.

**Observation RQ2-1:** Controlled cue removal produces the clearest repair-success degradation when both reproduction steps and stack traces are removed. This condition yields a significant success-rate decline ($\chi^2 = 9.71$, $p = 0.0018$), and GPT-4.1 drops from 42 successful repairs in the original setting to 29 after both cues are removed, a 8.9% decrease. Rephrasing the same cues has a smaller and less consistently significant effect, indicating that missing diagnostic content matters more than surface wording changes.

*4.2.2* ***Impact on Pre-execution Self-Assessment.*** Beyond repair success, we evaluate how cue removal and rephrasing affect the model's pre-execution self-assessments, the two raw judgments from which pre-execution confidence is derived: *Problem Specification* and *Task Difficulty*. Figure 10 shows that well-specified ratings remain relatively high even when key diagnostic cues are removed. In contrast, Figure 11 reveals that rephrasing information increased the agent's perceived difficulty, indicating that explicit judgments are more responsive to changes in surface phrasing than to the actual completeness of the input. Notably, both Claude Sonnet 4 and GPT-4.1 exhibited substantial shifts in self-assessment: their *Problem Specification* ratings dropped from Level 0 to Level 1 by 48.9 % and 40.1 %, respectively, when both cues were removed. While both models continued to label the tasks as relatively clear, this shift suggests partial responsiveness rather than complete insensitivity: the ratings move in the expected direction, but not sharply enough to provide a strong warning signal for the observed repair-success loss.

For the implicit uncertainty signal, we assessed the LLM's internal uncertainty using black-box estimation techniques and statistical analysis. Our results show that the largest *Eccentricity (ECC)* fluctuation between original and ablated reports is 4.3 % (less than 5 %). Paired $t$-tests ($p > 0.1$) and overlapping 95 % confidence intervals (CIs) confirm no significant differences. Yet these fluctuations were not significantly correlated with the actual presence or absence of diagnostic cues. This means ECC does not provide evidence that the model becomes reliably more uncertain when diagnostic cues are removed.

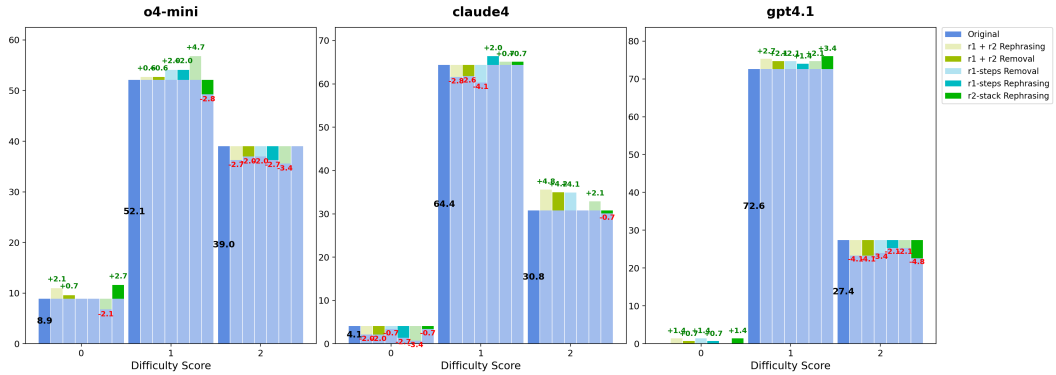


Fig. 11. Distribution of **Task Difficulty** self-assessment scores across diagnostic information conditions. The y-axis refers to the percentage of judgments at each score level. Bars compare original, rephrased, and removed cue settings (`r1-steps`, `r2-stack`) for *o4-mini*, *Claude Sonnet 4*, and *GPT-4.1*.

Table 6. Spearman rank correlation for change-based alignment. Following §3.5.1, $s$ and $d$ denote the model's *Problem Specification* and *Task Difficulty* self-assessment scores, respectively, and $G$ denotes the binary repair outcome ($G = 1$ for success). $\Delta s$, $\Delta d$, and $\Delta G$ are measured relative to the original problem statement, and $-\Delta G$ indicates perturbation-induced performance degradation. We report $\rho_S = \text{Spearman}(\Delta s, -\Delta G)$ for specification-shift alignment and $\rho_D = \text{Spearman}(\Delta d, -\Delta G)$ for difficulty-shift alignment. Higher values indicate that self-assessment shifts more consistently track performance degradation.

| Variant | o4-mini | | Claude Sonnet 4 | | GPT-4.1 | |
|---|---|---|---|---|---|---|
| | $\rho_S$ | $\rho_D$ | $\rho_S$ | $\rho_D$ | $\rho_S$ | $\rho_D$ |
| `r1_steps` Removal | 0.0541 | 0.0249 | 0.2334 | -0.0396 | 0.0707 | 0.0261 |
| `r2_stack` Removal | -0.0174 | 0.1277 | 0.1099 | 0.1895 | -0.0058 | 0.0089 |
| `r1_steps + r2_stack` Removal | 0.2797 | 0.1683 | 0.3299 | 0.2614 | 0.1973 | 0.0180 |
| `r1_steps` Rephrasing | 0.0499 | 0.0861 | 0.1397 | 0.0481 | 0.0041 | -0.0303 |
| `r2_stack` Rephrasing | 0.2382 | 0.1198 | 0.1173 | -0.0631 | 0.0820 | 0.0411 |
| `r1_steps + r2_stack` Rephrasing | 0.1797 | -0.0210 | 0.1285 | -0.0556 | 0.0966 | 0.0081 |
| **Mean across variants** | **0.1307** | **0.0843** | **0.2004** | **0.0889** | **0.0742** | **0.0120** |

Together, the explicit self-assessment and ECC results indicate an asymmetry: repair success can degrade when diagnostic content is removed, but pre-execution self-assessment and implicit uncertainty shift more weakly than the repair outcome.

**Observation RQ2-2:** Pre-execution self-assessment changes less sharply than repair success under cue removal. When both cues are removed, Claude Sonnet 4 and GPT-4.1 shift many *Problem Specification* ratings from Level 0 to Level 1 (48.9% and 40.1%, respectively), but the ratings still remain within the clearer side of the scale. The implicit uncertainty proxy changes even less: the largest ECC fluctuation is 4.3%, paired $t$-tests are not significant ($p > 0.1$), and 95% confidence intervals overlap. Thus, self-assessment and implicit uncertainty are partially responsive but do not provide a strong warning signal for the observed performance loss.

*4.2.3* ***Change-based Alignment Analysis.*** Following the definitions in §3.5.1, Table 6 reports how changes in *Problem Specification* ($\Delta s$) and *Task Difficulty* ($\Delta d$) self-assessment scores align with perturbation-induced performance degradation ($-\Delta G$). The results show that perturbation-induced shifts in self-assessment are only weakly aligned with the corresponding shifts in repair performance. For *Problem Specification*, the strongest specification-shift alignment consistently appears under `r1_steps + r2_stack` Removal, where $\rho_S$ is highest for all three judges. However, even the largest value (Claude Sonnet 4, $\rho_S = 0.3299$) indicates only limited alignment. Averaged across variants, Claude Sonnet 4 shows the highest mean specification-shift alignment ($\rho_S = 0.2004$), followed by o4-mini ($\rho_S = 0.1307$) and GPT-4.1 ($\rho_S = 0.0742$), but all remain modest in magnitude.

For *Task Difficulty*, difficulty-shift alignment is weaker still. Most $\rho_D$ values are close to zero, several are negative, and no stable pattern emerges across perturbation variants or judges. This suggests that increases in self-assessed difficulty do not consistently track the corresponding degradation in repair performance under cue removal or rephrasing. Overall, the change-based alignment results indicate that perturbation-induced performance degradation is only partially reflected in pre-execution self-assessment shifts.

In practical terms, these $\rho$ values indicate weak monotonic tracking: self-assessment changes sometimes move in the expected direction, especially for specification under both-cue removal, but not strongly enough to serve as reliable proxies for repair degradation.

**Observation RQ2-3:** Change-based alignment between self-assessment shifts and repair degradation is weak. Specification shifts are most aligned under both-cue removal, but even the largest value is limited (Claude Sonnet 4, $\rho_S = 0.3299$). Mean specification-shift alignment remains modest across variants: 0.2004 for Claude Sonnet 4, 0.1307 for o4-mini, and 0.0742 for GPT-4.1. Difficulty-shift alignment is weaker still, with mean $\rho_D$ values of 0.0889, 0.0843, and 0.0120. These values indicate that pre-execution ratings only weakly track perturbation-induced repair degradation.

**Finding for RQ2:** Observations RQ2-1 to RQ2-3 show a confidence-competence alignment gap under controlled diagnostic-cue perturbations. Observation RQ2-1 shows that removing both reproduction steps and stack traces can produce a practically visible repair-success drop (GPT-4.1: 42 to 29 successes, 8.9% decrease; $\chi^2 = 9.71$, $p = 0.0018$). Observation RQ2-2 shows that pre-execution self-assessment and implicit uncertainty respond more weakly than repair success, with ECC changes below 5% and no significant paired-test differences. Observation RQ2-3 shows that rating shifts only weakly track performance degradation, with mean $\rho_S \leq 0.2004$ and mean $\rho_D \leq 0.0889$ across models. Thus, agents rely on diagnostic content for successful repair, but their pre-execution self-assessments are only partially responsive and are not reliable proxies for the practical loss of repair competence.

## 4.3 RQ3: Behavioral Response to Input Perturbations

Building on the findings of RQ1 and RQ2, we next examine how repair trajectories change when diagnostic cues are removed or rephrased. While earlier analyses focused on judgment shifts and outcome-level performance, RQ3 takes a trajectory-level view and analyzes how operation patterns, testing behavior, and recovery behavior respond under perturbed inputs. We report both statistical reliability and effect-size magnitude because trajectory features are noisy and small statistical differences may not imply large behavioral changes. Following Table 3, we use a fixed-dimensional feature representation only as a common measurement schema; our analysis compares changes in

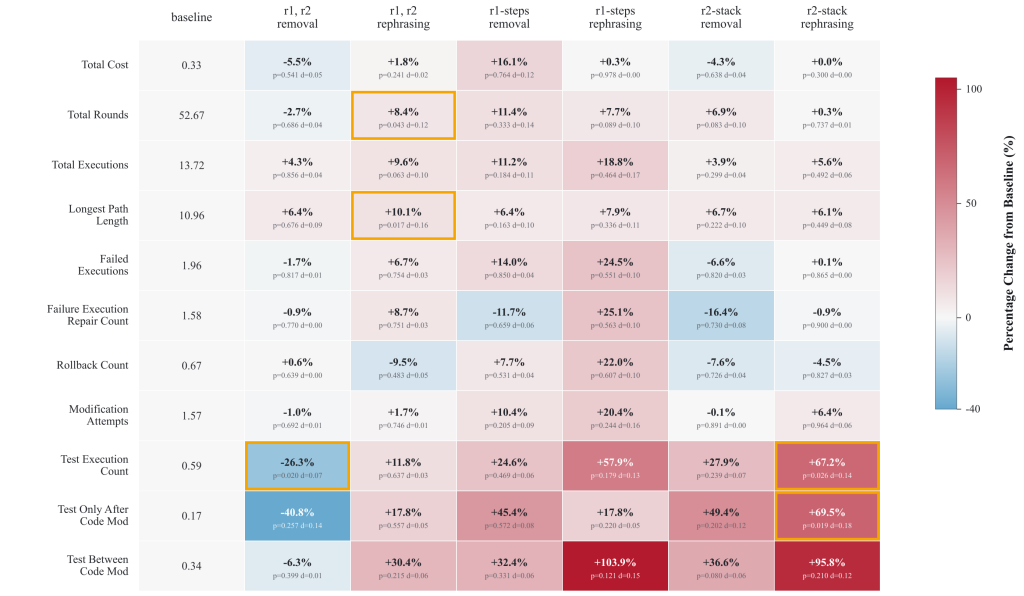


Fig. 12. Heatmap showing the percentage change in trajectory-level behavioral features under each information-ablation condition, relative to the mean feature value in the first "baseline" column. Thus, each row should be read against its baseline value: the remaining columns report how that feature changes under cue removal or rephrasing. Feature definitions are given in Table 3. Red indicates an increase, blue indicates a decrease, and a gold border indicates a statistically significant difference from the baseline condition under the Mann–Whitney U test ($p < 0.05$). Effect sizes are measured using Cohen's $d$ and are reported in the accompanying analysis.

feature values, not changes in the dimensionality of the feature vector. By extracting quantitative features from structured operation trees, we link fine-grained agent actions to outcomes, enabling direct attribution analysis.

*4.3.1* ***Behavioral Response Under Input Perturbation.*** Figure 12 summarizes how trajectory-level behavioral features change under diagnostic-information perturbations relative to the baseline condition. Overall, we observe that removing or rephrasing diagnostic cues does not simply reduce success rates; it also changes how agents allocate effort across execution, testing, and recovery behaviors.

**Execution Efficiency.** Rephrasing diagnostic information led to more extended execution trajectories, as reflected by increases in trajectory-length-related features. For example, under the *r1, r2 rephrasing* condition, we observed increases in both *total executions* (+9.6%, $p = 0.063$, $d = 0.10$) and *total rounds* (+8.4%, $p = 0.043$, $d = 0.12$). Similar increases also appear in *longest path length* under rephrasing conditions, suggesting deeper or more prolonged execution paths. In contrast, removal of diagnostic information had a smaller effect on these execution-length-related features.

**Testing Strategies.** Testing behavior is especially sensitive to perturbation type. Removing both diagnostic cues substantially reduces *test_execution_count* (−26.3%, $p = 0.020$, $d = 0.07$), indicating that agents perform less verification when key contextual grounding is absent. By contrast, rephrasing runtime-error information increases testing activity: under *r2-stack rephrasing*,

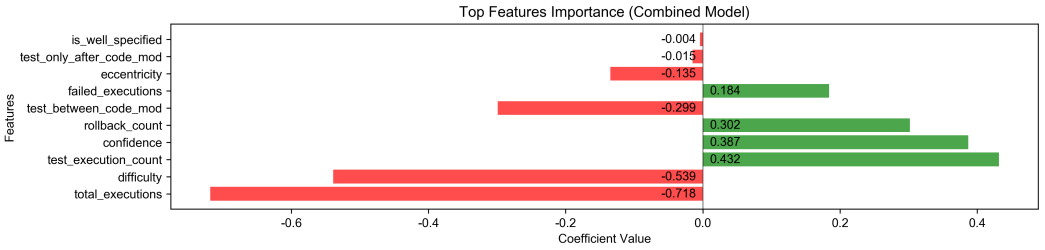


Fig. 13. Standardized logistic-regression coefficients for trajectory features used to predict repair outcome. Positive coefficients (Green) indicate association with successful repair, and negative coefficients (Red) indicate association with failure.

*test_execution_count* increases by +67.2% ($p = 0.026, d = 0.14$), and *test_between_code_mod* increases by +95.8% ($p = 0.015, d = 0.12$).

**Execution Error Handling.** In contrast to the stronger shifts observed for execution length and testing, lower-level recovery features remain relatively stable. In particular, *rollback_count*, *failed_executions*, and *failure_exe_repair_count* do not change substantially across perturbation conditions ($p > 0.1$). Practically, this suggests that perturbations mainly affect how much agents search and verify, rather than changing local recovery routines.

**Observation RQ3-1:** Diagnostic-information perturbations produce measurable but generally small trajectory-level behavioral shifts. Under *r1, r2 rephrasing*, total executions increase by 9.6% ($p = 0.063, d = 0.10$) and total rounds increase by 8.4% ($p = 0.043, d = 0.12$), indicating a modest tendency toward longer trajectories. Testing behavior changes more directionally: removing both cues reduces *test_execution_count* by 26.3% ($p = 0.020, d = 0.07$), while *r2-stack* rephrasing increases *test_execution_count* by 67.2% ($p = 0.026, d = 0.14$) and *test_between_code_mod* by 95.8% ($p = 0.015, d = 0.12$).

*4.3.2* ***Logistic Regression Analysis.*** To examine which trajectory patterns are associated with task outcomes, we performed an attribution analysis by training a logistic regression model on the extracted trajectory features (Table 3) to predict whether an agent successfully resolved an issue. The importance of the resulting characteristics is shown in Figure 13. The learned coefficients should be interpreted as outcome associations within the observed trajectories, not as causal effects of individual actions.

**Positive Associations.** Higher *rollback_count* and more frequent *test_execution_count* are positively associated with successful repair. This suggests that successful repairs tend to involve more verification and rollback behavior, without implying that either action alone causes success.

**Negative Associations.** Longer and more repetitive trajectories, reflected in features such as *total_executions* and *test_between_code_mod*, are negatively associated with success. Higher self-assessed *Task Difficulty* is also associated with failure. These patterns suggest that extended but unproductive execution loops are often a sign of ineffective behavioral response rather than productive exploration.

Together, these outcome-associated features distinguish productive response from unproductive effort: more activity is not necessarily better when it appears as long, repetitive execution rather than targeted verification or rollback.

**Observation RQ3-2:** Outcome-associated trajectory features distinguish productive response from unproductive effort. In the logistic-regression analysis, *rollback_count* and *test_execution_count* have positive associations with successful repair, whereas *total_executions*, *test_between_code_mod*, and higher self-assessed *Task Difficulty* are associated with failure. Thus, success is linked more to verification and rollback behavior than to simply longer or more repetitive execution.

**Finding for RQ3:** Observations RQ3-1 and RQ3-2 show that agents respond behaviorally to diagnostic-cue perturbations, but the response is uneven and not reliably corrective. Observation RQ3-1 shows modest trajectory-level shifts: rephrasing can increase execution effort (e.g., +9.6% total executions and +8.4% total rounds under *r1, r2 rephrasing*), while cue removal can suppress verification (-26.3% *test_execution_count* under both-cue removal). Observation RQ3-2 shows that successful repairs are associated with verification and rollback behavior, whereas longer and more repetitive trajectories are associated with failure. Therefore, RQ3 shows that perturbations measurably reshape trajectories, but increased activity alone does not imply effective adaptation.

### 4.4 RQ4: Toward Self-Judgment-Based Correction

The preceding analyses in RQ2 and RQ3 show that input perturbations can degrade repair success and reshape agent trajectories, but they do not indicate whether a completed attempt should be trusted. We therefore turn to RQ4 and examine whether post-execution self-judgment, combined with observable trajectory evidence, can help detect likely failed repair attempts and inform whether a candidate patch should be accepted, further validated, or revised.

*4.4.1* ***Characterizing Post-execution Self-Judgment.*** We begin by examining how *Post-execution Self-Judgment (PSJ)* changes after observing the full execution trajectory. A preliminary analysis on a 10% sample showed that the PSJ assessment remains highly stable during the final stages of a trajectory (variation $< 5\%$ at major turning points). This allowed us to simplify the analysis by focusing on the final post-execution judgment as a reliable summary of trajectory-level self-assessment, substantially reducing computational cost. Figure 14 groups results by cue-removal and cue-rephrasing settings only as a diagnostic check, to verify that the observed pattern is not driven by one intervention setting. The results show that observing the completed repair process systematically changes the agent's retrospective assessment.

**An Upward Shift in Perceived Specification.** After observing the execution trace, the proportion of issues judged as "well-specified" increases. This suggests that trajectory evidence can be interpreted as additional contextual grounding, even when the observed actions are ultimately unproductive. In other words, post-execution judgment tends to revise specification upward after seeing the repair process unfold.

**A Downward Shift in Perceived Difficulty.** PSJ also tends to revise task difficulty downward. Moreover, this revised difficulty assessment becomes associated with the observed execution process rather than only the original problem statement. This indicates that post-execution difficulty judgments partly reflect retrospective interpretation of how the repair process unfolded.

**A Simultaneous Drop in Confidence.** Most notably, the upward shift in perceived specification and the downward shift in perceived difficulty are accompanied by a decline in confidence. Thus, post-execution judgment does not move uniformly in a more optimistic direction. Instead, the combination of more favorable specification and difficulty ratings with lower confidence suggests


Fig. 14. Changes in PSJ post-execution evaluations relative to the initial self-assessments. The plots show shifts in ratings of *Problem Specification*, *Task Difficulty*, and *confidence* after observing the full execution trajectory. Results are grouped by cue-removal and cue-rephrasing settings only to check whether the observed shifts are consistent across interventions. Green bars indicate increases, and red bars indicate decreases.

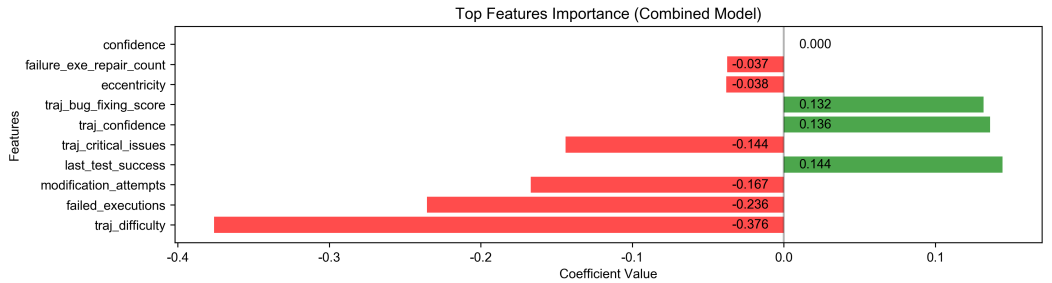


Fig. 15. Relative importance of PSJ-derived and behavioral features in the combined failure-detection model.

that the trajectory introduces signals that weaken certainty even when categorical judgments become more optimistic.

**Observation RQ4-1:** Post-execution self-judgment shifts in internally divergent directions after the agent observes its repair trajectory. Specification is often revised upward and difficulty downward, while confidence declines. This cross-dimension inconsistency means that the agent's categorical assessment becomes more favorable even as its confidence weakens, so the useful signal comes from the pattern of disagreement among PSJ dimensions rather than from any single revised rating.

*4.4.2* ***Failure Detection Analysis.*** As suggested by Observation RQ4-1 (§4.4.1), post-execution self-judgment signals alone are not sufficiently reliable for failure detection. Cross-dimension inconsistency refers to disagreement among PSJ dimensions, especially cases where the agent retrospectively rates the task as better specified or easier while simultaneously lowering its confidence. We therefore combine trajectory-level behavioral features from RQ3 with PSJ-derived features to test whether post-execution self-judgment provides complementary predictive value beyond behavior alone. Following §3.6.3, we fit logistic regression models under three settings: (1) PSJ-only (direct binary judgment/features), (2) behavioral features only, and (3) behavioral + PSJ features. Our goal is not to optimize absolute predictive performance, but to evaluate whether

Table 7. Comparison of predictive performance across failure-detection methods for completed repair attempts. *PSJ*: Post-execution Self-Judgment; AUC: area under the ROC curve; Acc.: accuracy; Prec.: precision.

| Method | AUC | Acc. | Prec. | Recall |
|---|---|---|---|---|
| Binary Judgment (*PSJ* Features Only) | 0.61 | 0.59 | 0.63 | 0.68 |
| Logistic Regression (Behavioral Features Only) | 0.70 | 0.75 | 0.73 | 0.75 |
| Logistic Regression (Behavioral + *PSJ* Features) | 0.74 | 0.77 | 0.77 | 0.77 |

explicit post-execution self-judgment signals improve discriminability between successful and failed completed repair attempts.

Table 7 reports the predictive performance. PSJ-only achieves an AUC of 0.61, while behavioral features alone reach 0.70. Importantly, augmenting behavioral features with PSJ features increases AUC to 0.74, with consistent improvements in accuracy, precision, and recall. In practical terms, PSJ alone is a weak failure detector, but the improvement from behavioral-only AUC 0.70 to behavioral+PSJ AUC 0.74 indicates that PSJ contributes complementary information not captured by trajectory behavior alone.

Figure 15 further clarifies the source of improvement. Several PSJ-derived features, including *traj_difficulty*, *traj_critical_issues*, and *traj_bug_fixing_score*, emerge as outcome-associated signals alongside behavioral features. Notably, trajectory-conditioned assessments, and their shifts relative to the agent's pre-execution ratings, contribute more than absolute categorical ratings from the original problem statement. This suggests that post-execution self-judgment provides complementary discriminative information beyond trajectory behavior alone.

**Observation RQ4-2:** Post-execution self-judgment is weak as a stand-alone failure detector, but it improves discriminability when combined with behavioral trajectory features. PSJ-only reaches AUC 0.61, compared with 0.70 for behavioral features alone. Adding PSJ features to behavioral features increases AUC to 0.74 and improves accuracy from 0.75 to 0.77, precision from 0.73 to 0.77, and recall from 0.75 to 0.77. This indicates that PSJ provides complementary evidence for failure detection rather than a reliable stand-alone signal.

**Finding for RQ4:** Observations RQ4-1 and RQ4-2 show that post-execution self-judgment is useful as auxiliary evidence, not as a stand-alone failure detector. Observation RQ4-1 shows that PSJ contains a cross-dimension inconsistency signal: specification and difficulty ratings become more favorable while confidence declines. Observation RQ4-2 shows that PSJ-only detection is weak (AUC 0.61), but adding PSJ to behavioral features improves discriminability over behavior alone (AUC 0.74 vs. 0.70, with accuracy, precision, and recall also increasing). Thus, RQ4 suggests that self-judgment-based correction is a promising direction for providing complementary validation signals when deciding whether to accept, further validate, or revise a completed repair attempt.

### 4.5 Threats to Validity

Our study aims at a principled analysis of judgment, self-assessment, and behavioral response in LLM-based software repair, rather than leaderboard optimization. At the time of experimentation,

OpenHands was the top-ranked and most widely adopted open-source agent framework on SWE-bench. Although several stronger methods appeared shortly before submission, their reported gains were moderate. We therefore used OpenHands as a strong and representative baseline to support a fair, reproducible analysis of agent performance. This choice introduces scaffold dependence in RQ3: trajectory-level features such as rollback count, test frequency, and execution length may partly reflect OpenHands control logic rather than purely model-internal strategic adaptation. We did not disable individual scaffold modules, because search, inspection, editing, execution, and testing are jointly required for SWE-bench repair and their removal would change the task setting. We therefore mitigate this threat through six controlled input-perturbation variants and outcome-associated attribution analysis, while leaving cross-scaffold validation to future work.

Because our study relies on commercial black-box LLMs, internal interpretability methods are not applicable. Instead, we use two observable black-box signal sources: (1) explicit self-assessment signals, including specification and difficulty judgments elicited through an LLM-as-Judge setup; and (2) implicit uncertainty signals estimated through semantic consistency under paraphrased inputs. These signals do not reveal internal mechanisms directly, but they provide operationally measurable proxies for studying self-assessment and judgment–performance misalignment.

A further threat is potential mismatch between Judge and Agent perspectives. To reduce this risk, we used the same underlying LLM for both roles and additionally verified robustness across three different model families. We also used randomized label-order prompting for the LLM Judge to mitigate possible positional bias in judgment outputs.

Run-to-run randomness poses an additional threat to validity. Bjarnason et al. [6] show that single-run *pass@1* estimates in agentic SWE-bench evaluation can vary substantially, so small success-rate differences may reflect evaluation noise. We partly mitigate this risk through multi-round information-rephrasing interventions, which preserve diagnostic content while changing surface wording, and by checking whether the resulting patterns are consistent across three LLM families and across removal/rephrasing variants. Our conclusions therefore rely on consistent comparative patterns rather than isolated small differences. Nevertheless, rephrasing-based robustness checks do not fully substitute for many independent repeated runs of the same agent trajectory. Because exhaustive repetition over all models, issues, and intervention variants would require prohibitive computational cost, we acknowledge this limitation and leave large-scale repeated-run estimation to future work.

As another threat related to judgment comparison, human developer skill and experience may vary substantially. Not all developers approach issue resolution in the same way, and differences in expertise may affect how much clarification, iteration, or testing is needed during debugging. This means that some of the interaction patterns we discuss may depend not only on issue quality, but also on the capabilities of the developer or agent attempting the task. To reduce this threat, our analyses follow the SWE-bench and SWE-bench Verified setup, which frames difficulty relative to experienced software engineers and uses standardized benchmark instances rather than unconstrained human debugging sessions. Nevertheless, our results should be interpreted as characterizing behavior under this benchmark setting, rather than as claims about all developers or all debugging styles.

Finally, our findings may be sensitive to model choice and prompt formulation. To address this, we repeated the experiments with three state-of-the-art LLMs and consistently observed weak human–LLM judgment alignment (RQ1), weak pre-execution confidence–competence alignment under perturbation (RQ2), and similar trajectory-level response patterns (RQ3). We also evaluated rephrasing-based perturbations in addition to direct cue removal, allowing us to distinguish the effect of missing diagnostic information from surface-level wording variation. Across settings, cue removal produced the clearest and most consistent degradation in repair success, supporting the robustness of our central findings.

## 5 Discussion

Our study reveals two related but distinct forms of misalignment in agentic software repair: (1) weak alignment between human and LLM judgments of issue conditions, and (2) weak alignment between agents' pre-execution self-assessments and their realized repair competence. This section discusses the nature of this mismatch, the recurring patterns we observe in agent behavior under missing information, and the implications for future agent design.

### 5.1 The Judgment-Performance Problem: Confidence vs. Competence

A central finding of our study is a persistent *confidence–competence gap*: agents' pre-execution self-assessments remain comparatively optimistic even when their realized repair success is substantially reduced. When critical diagnostic cues such as reproduction steps or stack traces are removed, repair success drops significantly, yet the corresponding shifts in self-assessment remain comparatively weak.

This pattern suggests that current agents do not reliably translate degraded input conditions into appropriately degraded pre-execution self-assessments. Our implicit uncertainty analyses further indicate that response instability can increase under ambiguity, but these proxy signals are only partially reflected in explicit judgments. In other words, the issue is not only whether agents exhibit uncertainty, but whether that uncertainty is adequately surfaced and incorporated into downstream decision-making.

### 5.2 Systematic Biases in Information Processing

Our results reveal several recurring patterns in how agents behave when human-valued diagnostic cues are incomplete or perturbed.

**Optimistic Pre-execution Self-assessment Under Incomplete Information.** Compared with human annotators, LLM judges tend to rate issue reports as clearer and more actionable than supported by the available diagnostic information. Under cue removal, this leads to a weak relationship between self-assessment shifts and the actual decline in repair performance.

**Uneven sensitivity to diagnostic cues.** As motivated in §3.4.1 and examined in RQ1, diagnostic cues such as reproduction steps provide signals for assessing issue informativeness. However, agent self-assessments appear much less sensitive to the absence of these cues. This suggests that agents do not weight the same issue-report elements in the same way as human developers when estimating tractability.

**Partial but limited behavioral response.** At the trajectory level, agents respond when diagnostic cues are removed or rephrased: testing behavior changes, trajectories may lengthen, and execution patterns shift. However, these responses are often only partial and do not reliably recover the lost performance caused by missing information.

### 5.3 Implications for Agent Design

These findings suggest that improving agentic software repair is not only a matter of stronger execution or better search. It also requires better mechanisms for judging when the available information is sufficient, when confidence should be reduced, and when the agent should revise its course of action.

Rather than assuming that every issue should immediately enter a full repair pipeline, future systems should explicitly reason about issue tractability before execution. This includes assessing whether key diagnostic cues are present, whether the task appears sufficiently specified, and whether the agent's own signals indicate elevated failure risk. In this sense, more reliable repair agents should be not only execution-capable, but also *judgment-aware*.

Our RQ4 results further suggest that this is feasible. Although post-execution self-judgment alone is not sufficiently reliable as a stand-alone predictor, it provides complementary information beyond behavioral traces. This indicates that self-judgment-based correction may be a promising direction for deciding whether a completed repair attempt should be accepted, further validated, or revised.

### 5.4 Toward Judgment-Aware and Correction-Aware Repair Systems

Taken together, our findings support a shift in emphasis from pure automation toward *judgment-aware* and *correction-aware* repair systems. We highlight three practical directions.

**Diagnostic-information awareness.** Repair agents should explicitly detect whether critical cues are present before attempting resolution. Missing reproduction steps, missing stack traces, or other forms of underspecification should not be treated as ordinary inputs.

**Confidence–competence alignment.** Agents should be designed so that their pre-execution self-assessments better track actual repair success likelihood. This does not require assuming human-like internal states; rather, it requires more reliable operational alignment between explicit self-assessment signals and downstream performance.

**Trajectory-aware correction.** Agent trajectories already contain useful signals of failure risk. Systems should monitor these behavioral patterns during execution and combine them with post-execution self-judgment signals to support early intervention, fallback strategies, or requests for additional information.

## 6 Conclusion

This paper presents the first systematic empirical study of two judgment-related gaps in agentic software repair: the gap between human and LLM judgments of issue conditions, and the gap between agents' pre-execution self-assessments and downstream repair competence. Through an analysis of 1,699 real-world GitHub issues from SWE-bench, we find weak human–LLM judgment alignment, weak pre-execution confidence–competence alignment under cue perturbation, and distinct behavioral responses that do not necessarily translate into effective recovery. At the same time, post-execution self-judgment and behavioral signals retain useful discriminative value for distinguishing successful from failed runs.

Overall, our findings suggest that current failures in agentic software repair reflect not only execution limitations, but also persistent misalignment between issue-condition judgment, pre-execution self-assessment, and realized repair competence. We hope this work motivates more judgment-aware and correction-aware repair agents.

## Acknowledgments

This work is partially funded by the Dieter Schwarz Foundation and the Technical University of Munich – Institute for Advanced Study, Germany.

## References

[1] Vaibhav Aggarwal, Ojasv Kamal, Abhinav Japesh, Zhijing Jin, and Bernhard Schölkopf. 2025. Dars: Dynamic action re-sampling to enhance coding agent performance by adaptive tree traversal. *arXiv preprint arXiv:2503.14269* (2025).

[2] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. Swe-bench+: Enhanced coding benchmark for llms. *arXiv preprint arXiv:2410.06992* (2024).

[3] Abdulaziz Alshayban, Iftekhar Ahmed, and Sam Malek. 2020. Accessibility issues in android apps: state of affairs, sentiments, and ways forward. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 1323–1334.

[4] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the dangers of stochastic parrots: Can language models be too big?. In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*. 610–623.

[5] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What makes a good bug report? *(SIGSOFT '08/FSE-16)*. Association for Computing Machinery, New York, NY, USA, 308–318. doi:10.1145/1453101.1453146

[6] Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. 2026. On randomness in agentic evals. *arXiv preprint arXiv:2602.07150* (2026).

[7] Lili Bo, Wangjie Ji, Xiaobing Sun, Ting Zhang, Xiaoxue Wu, and Ying Wei. 2024. Chatbr: Automated assessment and improvement of bug report quality using chatgpt. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1472–1483.

[8] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 2188–2200. doi:10.1109/ICSE55347.2025.00157

[9] Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail?. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. https://openreview.net/forum?id=fAjbYBmonr

[10] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting missing information in bug descriptions. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering* (Paderborn, Germany) *(ESEC/FSE 2017)*. Association for Computing Machinery, New York, NY, USA, 396–407. doi:10.1145/3106237.3106285

[11] Mengzhuo Chen, Zhe Liu, Chunyang Chen, Junjie Wang, Boyu Wu, Jun Hu, and Qing Wang. 2025. Standing on the Shoulders of Giants: Bug-Aware Automated GUI Testing via Retrieval Augmentation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 825–846.

[12] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 8697–8727. doi:10.18653/v1/2025.acl-long.426

[13] Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao, et al. 2025. The danger of overthinking: Examining the reasoning-action dilemma in agentic tasks. *arXiv preprint arXiv:2502.08235* (2025).

[14] Yuanrui Fan, Xin Xia, David Lo, and Ahmed E Hassan. 2018. Chaff from the wheat: Characterizing and determining valid bug reports. *IEEE transactions on software engineering* 46, 5 (2018), 495–525.

[15] Sidong Feng and Chunyang Chen. 2024. Prompting is all you need: Automated android bug replay with large language models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[16] Yarin Gal and Zoubin Ghahramani. 2016. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*. PMLR, 1050–1059.

[17] Pavlina Wurzel Goncalves, Pooja Rani, Margaret-Anne Storey, Diomidis Spinellis, and Alberto Bacchelli. 2025. Code Review Comprehension: Reviewing Strategies Seen Through Code Comprehension Theories . In *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*. IEEE Computer Society, Los Alamitos, CA, USA, 589–601. doi:10.1109/ICPC66645.2025.00068

[18] D.M. Green and J.A. Swets. 1966. *Signal Detection Theory and Psychophysics*. Wiley.

[19] Jianjun Huang, Jianglei Nie, Yuanjun Gong, Wei You, Bin Liang, and Pan Bian. 2024. Raisin: Identifying rare sensitive functions for bug detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.

[20] Jirayus Jiarpakdee, Chakkrit Tantithamthavorn, and Ahmed E. Hassan. 2021. The Impact of Correlated Metrics on the Interpretation of Defect Models. *IEEE Trans. Softw. Eng.* 47, 2 (Feb. 2021), 320–331. doi:10.1109/TSE.2019.2891758

[21] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VTF8yNQM66

[22] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221* (2022).

[23] Asher Koriat et al. 2006. *Metacognition and consciousness*. Institute of Information Processing and Decision Making, University of Haifa.

[24] Sandeep Kaur Kuttal, Bali Ong, Kate Kwasny, and Peter Robe. 2021. Trade-offs for substituting a human with an agent in a pair programming context: the good, the bad, and the ugly. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–20.

[25] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2026. LLMs Get Lost In Multi-Turn Conversation. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=VKGTGGcwl6

[26] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. 2017. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems* 30 (2017).

[27] Xun Liang, Jiawei Yang, Yezhaohui Wang, Chen Tang, Zifan Zheng, Shichao Song, Zehao Lin, Yebin Yang, Simin Niu, Hanyu Wang, et al. 2025. Surveyx: Academic survey automation via large language models. *arXiv preprint arXiv:2502.14776* (2025).

[28] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 3214–3252. doi:10.18653/v1/2022.acl-long.229

[29] Zhen Lin, Shubhendu Trivedi, and Jimeng Sun. 2024. Generating with confidence: Uncertainty quantification for black-box large language models. *Transactions on Machine Learning Research* (2024).

[30] Jinyi Liu, Yan Zheng, Rong Cheng, Qiyu Wu, Wei Guo, Fei Ni, Hebin Liang, Yifu Yuan, Hangyu Mao, Fuzheng Zhang, et al. 2025. From chaos to order: The atomic reasoner framework for fine-grained reasoning in large language models. *arXiv preprint arXiv:2503.15944* (2025).

[31] Xiaoou Liu, Tiejin Chen, Longchao Da, Chacha Chen, Zhen Lin, and Hua Wei. 2025. Uncertainty Quantification and Confidence Calibration in Large Language Models: A Survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) *(KDD '25)*. Association for Computing Machinery, New York, NY, USA, 6107–6117. doi:10.1145/3711896.3736569

[32] Yizhou Liu, Pengfei Gao, Xinchen Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024. Marscode agent: Ai-native automated bug fixing. *arXiv preprint arXiv:2409.00899* (2024).

[33] Ian R. McKenzie, Alexander Lyzhov, Michael Martin Pieler, Alicia Parrish, Aaron Mueller, Ameya Prabhu, Euan McLean, Xudong Shen, Joe Cavanagh, Andrew George Gritsevskiy, Derik Kauffman, Aaron T. Kirtland, Zhengping Zhou, Yuhui Zhang, Sicong Huang, Daniel Wurgaft, Max Weiss, Alexis Ross, Gabriel Recchia, Alisa Liu, Jiacheng Liu, Tom Tseng, Tomasz Korbak, Najoung Kim, Samuel R. Bowman, and Ethan Perez. 2023. Inverse Scaling: When Bigger Isn't Better. *Transactions on Machine Learning Research* (2023). https://openreview.net/forum?id=DwgRm72GQF Featured Certification.

[34] Thomas O Nelson. 1996. Consciousness and metacognition. *American psychologist* 51, 2 (1996), 102.

[35] A. Newell and H.A. Simon. 1972. *Human Problem Solving*. Prentice-Hall.

[36] OpenAI. 2024. *Introducing SWE-bench Verified*. https://openai.com/index/introducing-swe-bench-verified/ Accessed: 2025-07-13.

[37] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. 2025. Training Software Engineering Agents and Verifiers with SWE-Gym. In *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=Cq1BNvHx74

[38] Debalina Ghosh Paul, Hong Zhu, and Ian Bayley. 2024. Benchmarks and metrics for evaluations of code generation: A critical review. In *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*. IEEE, 87–94.

[39] Peter Robe and Sandeep Kaur Kuttal. 2022. Designing pairbuddy—a conversational agent for pair programming. *ACM Transactions on Computer-Human Interaction (TOCHI)* 29, 4 (2022), 1–44.

[40] A.H. SCHOENFELD. 1985. *Mathematical Problem Solving*. Elsevier Science.

[41] Yanqi Su, Zhenchang Xing, Chong Wang, Chunyang Chen, Sherry Xu, Qinghua Lu, and Liming Zhu. 2025. Automated Soap Opera Testing Directed by LLMs and Scenario Knowledge: Feasibility, Challenges, and Road Ahead. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 757–778.

[42] Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. 2026. Interactive Agents to Overcome Underspecificity in Software Engineering. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=X2yzXtH4wp

[43] Haoran Wang, Zhenyu Hou, Yao Wei, Jie Tang, and Yuxiao Dong. 2025. SWE-Dev: Building Software Engineering Agents with Training and Inference Scaling. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 3742–3761. doi:10.18653/v1/2025.findings-acl.193

[44] Huacan Wang, Ziyi Ni, Shuo Zhang, Shuo Lu, Sen Hu, Ziyang He, Chen Hu, Jiaye Lin, Yifu Guo, Yuntao Du, and Pin Lyu. 2025. RepoMaster: Autonomous Exploration and Understanding of GitHub Repositories for Complex Task Solving. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=aSfBbhUJAa

[45] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=OJd3ayDDoF

[46] Zhijie Wang, Zijie Zhou, Yuheng Huang Da Song, Shengmai Chen, Lei Ma, and Tianyi Zhang. 2025. Towards understanding the characteristics of code generation errors made by large language models. In *Proceedings of the IEEE/ACM 47th International Conference on software Engineering (ICSE'25)*.

[47] Miao Xiong, Zhiyuan Hu, Xinyang Lu, YIFEI LI, Jie Fu, Junxian He, and Bryan Hooi. 2024. Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=gjeQKFxFpZ

[48] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[49] Xixian Yong, Xiao Zhou, Yingying Zhang, Jinlin Li, Yefeng Zheng, and Xian Wu. 2025. Think or Not? Exploring Thinking Efficiency in Large Reasoning Models via an Information-Theoretic Lens. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=DpOSndSOZz

[50] Mingyue Yuan, Jieshan Chen, Zhenchang Xing, Aaron Quigley, Yuyu Luo, Tianqi Luo, Gelareh Mohammadi, Qinghua Lu, and Liming Zhu. 2025. DesignRepair: Dual-Stream Design Guideline-Aware Frontend Repair with Large Language Models. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering* (Ottawa, Ontario, Canada) *(ICSE '25)*. IEEE Press, 2483–2494. doi:10.1109/ICSE55347.2025.00109

[51] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (stop): Recursively self-improving code generation. In *First Conference on Language Modeling*.

[52] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating Agentic Workflow Generation. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=z5uVAKwmjf

[53] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=GazlTYxZss

[54] Yuze Zhao, Tianyun Ji, Wenjun Feng, Zhenya Huang, Qi Liu, Zhiding Liu, Yixiao Ma, Kai Zhang, and Enhong Chen. 2025. Unveiling the Magic of Code Reasoning through Hypothesis Decomposition and Amendment. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=kN25ggeq1J

## A Diagnostic Cue Annotation Schema

```
The following dimensions are used to annotate diagnostic cues in the problem
     statements of issue reports and in developer hints.

For Questions 1-7 and 9, use the following labels:
1 = Yes
0 = No
-1 = Cannot determine

If a question asks for a link, path, commit ID, pull request number, or other
     resource identifier, record the exact value when available.

General rules:
(1) Assign 1 only when the evidence is explicitly present in the provided text or
     attached resource.
(2) Assign 0 when the relevant cue is not provided.
(3) Assign -1 when the presence of the cue cannot be determined from the available
     material, including cases where it is unclear whether the text provides
     reproducibility evidence or runtime failure evidence.
(4) The questions are not mutually exclusive. For example, reproducibility evidence
      and runtime failure evidence may both be present.
```

```
A. Problem Statement Analysis (Questions 1-5)

Q1. Reproducibility Evidence.
Does the problem statement include either step-by-step reproduction instructions or
      executable test code/command, and also state the expected behavior or
     expected result?
Both the reproduction procedure/code and the expected behavior/result must be
     present.

Q2. Runtime Failure Evidence.
Does the problem statement include concrete evidence of an actual runtime failure,
     such as executable code or a command that triggers the failure, together with
     the exact error message, test failure output, or stack trace?
This must reflect an observed execution failure, not merely a description of an
     incorrect result or expected behavior.

Q3. Fix Code Provided.
Does the problem statement explicitly provide actual code, patch, or diff that
     directly implements or specifies the fix?
This must include real source code or a concrete code change. A natural-language
     suggestion alone does not count.

Q4. Multimedia Support.
Does the problem statement include any supporting multimedia evidence, such as
     images, videos, or audio recordings?
If yes, specify the resource path or link.

Q5. Linked Documentation.
Does the problem statement reference any external documentation that may assist in
     understanding or solving the issue, such as API documentation, official
     documentation, or design documentation?
If yes, specify the link or path.

B. Developer Response Analysis (Questions 6-9)

Q6. Reproducibility Evidence in Response.
Does the developer response include either step-by-step reproduction instructions
     or executable test code/command, and also state the expected behavior or
     expected result?
Use the same criteria as in Q1.

Q7. Runtime Failure Evidence in Response.
Does the developer response include concrete evidence of an actual runtime failure,
      such as executable code or a command that triggers the failure, together with
      the exact error message, test failure output, or stack trace?
Use the same criteria as in Q2.

Q8. Solution Format.
What kind of solution is described or referenced in the developer response?

Use one or more of the following identifiers:
SOL_CODE: direct code solution, patch, or diff
SOL_COMMIT: commit ID reference
SOL_PR: pull request reference
SOL_LINK: direct source file or code link
SOL_DOC: linked documentation relevant to the solution
SOL_NONE: no explicit solution artifact or solution reference provided

If applicable, provide identifiers with values using the following format:
SOL_COMMIT:a1b2c3d
```

```
SOL_PR:@1234
SOL_LINK:file.py@L20
[SOL_CODE, SOL_PR:@1234]

Additional rules for Q8:
(1) Use multiple identifiers if multiple solution forms are present.
(2) Use SOL_CODE only when actual code, patch, or diff is shown.
(3) Use SOL_DOC for documentation links, and use SOL_LINK for direct code or file
     links.
(4) Use SOL_NONE only when no explicit solution artifact or solution reference is
     provided.

Q9. Multimedia Support in Response.
Does the developer response include any supporting multimedia evidence, such as
     images, videos, or audio?
If yes, specify the resource path or link.
```

## B LLM-as-Judge Prompt for Reusing the Human Annotation Schema

To align our LLM-as-judge setup with the human annotations released in SWE-bench Verified, we directly reuse the same annotation structure in the prompt. Specifically, the judge is instructed to evaluate each issue along the same dimensions as the OpenAI human annotation schema: *Problem Specification*, *Task Difficulty*, and *Confidence*. The prompt presents the *Problem Statement*, the *Gold Patch*, and the corresponding *Test Patch*, and asks the model to produce a categorical rating together with a free-text explanation. This design serves two purposes. First, it keeps the elicited LLM judgments structurally comparable to the human annotations, which supports our human–LLM judgment alignment analyses. Second, by reusing the same question wording and scale definitions, it reduces avoidable prompt variation and makes the annotation protocol more transparent and reproducible. In our setup, we further provide few-shot examples through the `example_content` parameter to better align the rating scale with human judgments. These examples are taken from the original OpenAI human annotation schema and serve as the same score-level anchors used by the human annotators.

Notably, the original human annotation schema also includes a 1–5 *Confidence* rating. We do not use this confidence rating in our experiments, because it is inherently subjective and may vary substantially across annotators or models. Instead, we focus on the more operationally grounded *Problem Specification* and *Task Difficulty* dimensions to characterize pre-execution self-assessment. These dimensions are more directly tied to task tractability and downstream repair behavior. The core prompt template used in our implementation is shown below.

```
def system_prompt(file_content, example_content, Gold_Patch, Test_Patch):
    prompt = [
        {
            "role": "system",
            "content": "You are an experienced software engineer tasked with
                 judging whether issue descriptions are well-specified and suitable
                  for use in our benchmark, and assessing their difficulty."
        },
        {
            "role": "user",
            "content": f"""
            We have a dataset of GitHub issues from various open-source Python
                 repositories. Each issue comes with a PR that successfully solves
                 the issue described. Each PR consists of 2 parts:
```

```
(1) code that resolves the issue
(2) changes to the test files of the repository, which check whether
     the issue has been resolved.

We intend to use samples in this dataset as a benchmark for coding
     ability: For each sample, we give an engineer the issue text and
     ask them to write code to resolve the issue (without revealing the
      solution from the original PR). Then, we apply the test files
     from the original PR to their code and run the tests to check
     whether their solution passes.

Importantly, this setup assumes that:
# The issue description is sufficiently well-specified to understand
     what the problem is, and what the correct solution should look
     like.

In this task, you will help to check those assumptions and identify
     which issue samples are suitable for use in our benchmark.


Please take a moment to read the issue description below.
IMPORTANT: Do not click through on any external links, and do not read
     the discussion below the GitHub issue. Our setup will only provide
      the main issue text as shown below, so please answer the
     questions on the basis of only the text shown here.

{file_content}

Section 1 - Issue Description
How well-specified is the issue text?
Imagine that you are an experienced software engineer who has been
     instructed to create a PR that successfully resolves the above
     GitHub issue. You have full access to the codebase, and can see
     the issue description as it is above. But you are not able to ask
     for clarification and would need to work exclusively from this
     information.

Is the issue description well-specified enough for a meaningful attempt
      at a solution?
Question 1.1
# 0: The issue is well-specified and it is clear what is required for a
      successful solution.
## Example: pylint #5201 {example_content[0]}
# 1: There are some blanks to fill in about the issue, but there is a
     sensible interpretation of what is required for a successful
     solution.
## Example: sympy #18030 {example_content[1]}
# 2: The issue is vague and there is room for ambiguity. It is unclear
     what a successful solution would look like.
## Example: scikit-learn #14520 {example_content[2]}
# 3: It is almost impossible to understand what you are being asked to
     do without further information.
## Example: pylint #5743 {example_content[3]}

Please explain your choice above. Include references to specific
     filenames, function/class names, or lines of code where relevant.
Question 1.2
[Free text, minimum 100 characters]

(1) code that resolves the issue
(Gold Patch of the issue resolved by the PR)
```

```
            {Gold_Patch}

            (2) changes to the test files of the repository, which check whether
                 the issue has been resolved.
            (Test Patch of the issue resolved by the PR)
            {Test_Patch}

            Section 2
            You will consider the GitHub issue again and answer a final set of
                 questions. You may navigate back to previous sections to see the
                 issue and patches.

            Difficulty
            We wish to understand how difficult this issue is to solve.
            How long would it take (for an experienced software engineer who had a
                 few hours to familiarize themselves with the codebase) to
                 understand the problem described in the GitHub issue, arrive at a
                 solution and write the code for a solution?
            Note: If the issue text was previously too ambiguous to solve at all,
                 you may assume that the problem has been clarified sufficiently
                 such that the high-level requirements for the solution are clear
                 ("what"), but the specifics about the solution are left to the
                 engineer to figure out ("how").

            Question 2.1
            # 0: <15 min fix
            ## e.g., a trivial change adding some assertions to a function
            # 1: 15 min - 1 hour
            ## e.g., a small change that requires a bit of thought
            # 2: 1-4 hours
            ## e.g., substantially rewriting a function or editing multiple files
            # 3: >4 hours
            # e.g., a very esoteric issue that clearly requires a substantial
                 amount of research to fix, changing >100 lines of code

            Question 2.2
            Explain why you chose the difficulty level above.

            Question 2.3
            Are there any other major issues that haven't been covered? i.e. Any
                 other reasons this sample should not be used in our setup for
                 evaluating coding ability.
            # 0: No
            # 1: Yes

            Question 2.4
            Do you have any other notes you wish to add? If you answered 'Yes' to
                 the previous question 2.3, please explain here.
            [Free text, minimum 100 characters]

            Question 2.5
            Confidence
            Overall, how confident are you in the answers you have given for this
                 sample?
            [1,2,3,4,5]
            """
        }
    ]
```

**Question 1.1 Score 0: Well-specified, requirements are clear**

*Example: pylint #5201*

````
ignore-paths: normalize path to PosixPath
### Current problem

In a project of mine, there is an entire directory, "dummy", that I want to exclude
     running pylint in.  I've added the directory name to the "ignore" option and
    it works great when used from the command line.

```toml
# Files or directories to be skipped. They should be base names, not paths.
ignore = [
  'dummy',
]
```

However, when using vscode, the full path is provided.  It calls pylint like this:

```
~\Documents\<snip>\.venv\Scripts\python.exe -m pylint --msg-template='{line},{
    column},{category},{symbol}:{msg} --reports=n --output-format=text ~\Documents
    \<snip>\dummy\file.py
```

In this case, the ignore rule doesn't work and vscode still reports errors.  So I
    decided to switch to the "ignore-paths" option.  The following works:

```toml
# Add files or directories matching the regex patterns to the ignore-list. The
# regex matches against paths.
ignore-paths = [
  '.*/dummy/.*$',
  '.*\\dummy\\.*$',
]
```

However, I need to duplciate each path, onces for Linux (/ as path separator) and
    the second for Windows (\ as path separator).  Would it be possible to
    normalize the paths (could use pathlib PosixPath) so that just the linux one
    would work on both systems?  Note also that vscode passes the full path, so
    starting the regex with a ^, like '^dummy/.*$', won't work.

### Desired solution

I'd like to be able to define the path only once in the "ignore-paths" settings.
    Even better would be to respect the "ignore" setting even for a path provided
    with the full path (just as if it was run from the command line).

```toml
# Add files or directories matching the regex patterns to the ignore-list. The
# regex matches against paths.
ignore-paths = [
  '.*/dummy/.*$',
]
```

### Additional context

_No response_
````

**Question 1.1 Score 1: Some blanks remain, but a sensible interpretation is possible**
*Example: sympy #18030*

````
interpolate could provide value instead of nan
```python
>>> y = (18,25,43,70,115)
>>> interpolate(y,5)
nan
```
Since the default x value for interpolation is `range(1, len(y)+1)` the
     interpolation at 5 could just return 115 instead of nan.
````

**Question 1.1 Score 2: Vague, successful solution is ambiguous**
*Example: scikit-learn #14520*

```
Copy param ignored in TfidfVectorizer
I was playing with vectorizers and I found this:

https://github.com/scikit-learn/scikit-learn/blob/
     ae16319626e2ca6ca0e54d4a5b83f73f817232aa/sklearn/feature_extraction/text.py#
     L1669

However that parameter is not used later in the method.

Here `copy=False` is used:

https://github.com/scikit-learn/scikit-learn/blob/
     ae16319626e2ca6ca0e54d4a5b83f73f817232aa/sklearn/feature_extraction/text.py#
     L1692

Is there anything I am missing?
```

**Question 1.1 Score 3: Insufficient information to understand the task**
*Example: pylint #5743*

```
Investigate #5495 (crash without a provided template)
See https://github.com/PyCQA/pylint/issues/5495#issuecomment-1011022169
```

## C Diagnostic-cue-specific Semantic Eccentricity

To make the implicit-uncertainty analysis directly comparable with the diagnostic-cue analysis, we report semantic eccentricity distributions after splitting issue samples by cue presence. Each figure compares cue-absent (=0) and cue-present (=1) samples for *o4-mini*, *GPT-4.1*, and *Claude Sonnet 4*.

Overall, these diagnostic-cue-specific distributions are consistent with the main analysis in §4.1.5. Across models and cue types, cue-present and cue-absent samples exhibit broadly overlapping eccentricity distributions, with no stable visual separation indicating that the presence of a diagnostic cue systematically corresponds to higher or lower implicit uncertainty.

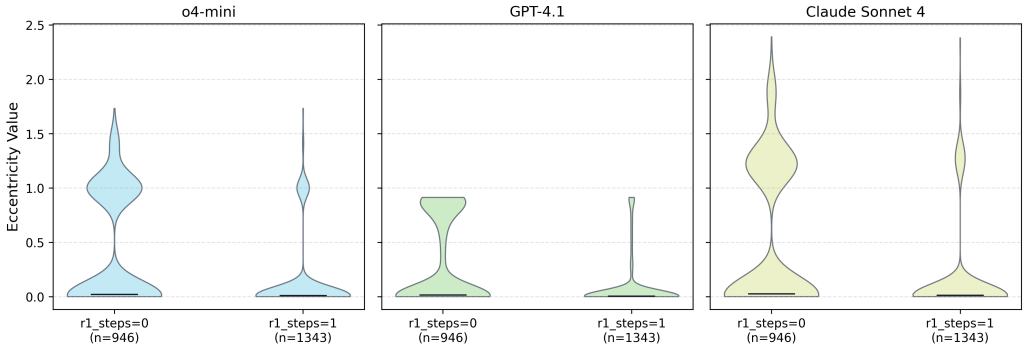


Fig. 16. Semantic eccentricity grouped by the presence of `r1-steps`.

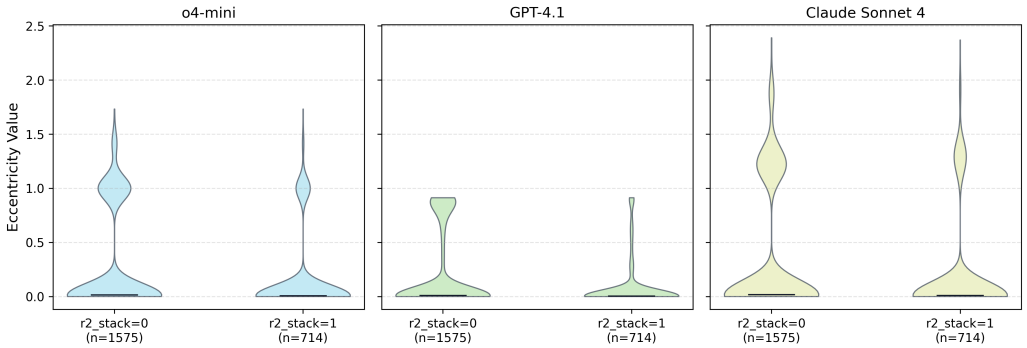


Fig. 17. Semantic eccentricity grouped by the presence of `r2-stack`.

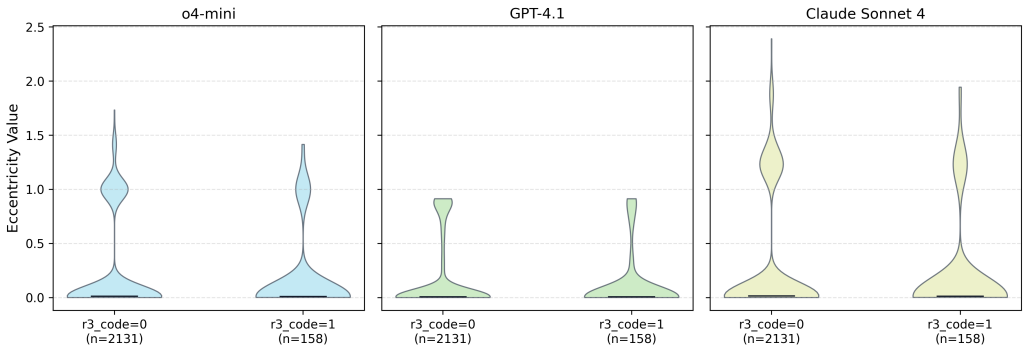


Fig. 18. Semantic eccentricity grouped by the presence of `r3-code`.

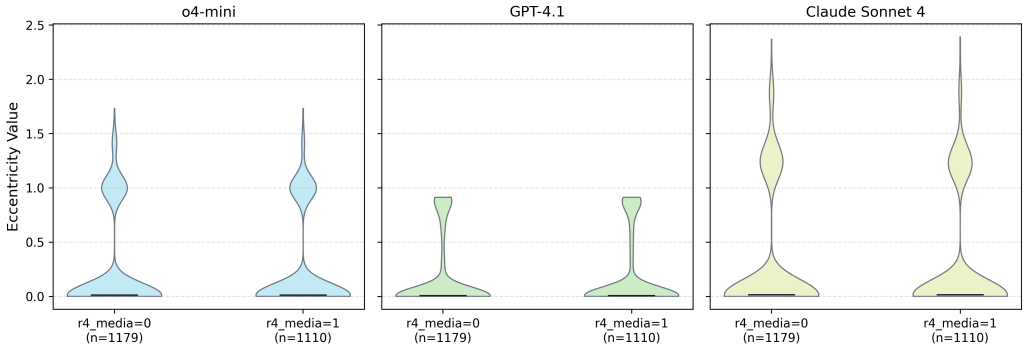


Fig. 19. Semantic eccentricity grouped by the presence of `r4-media`.

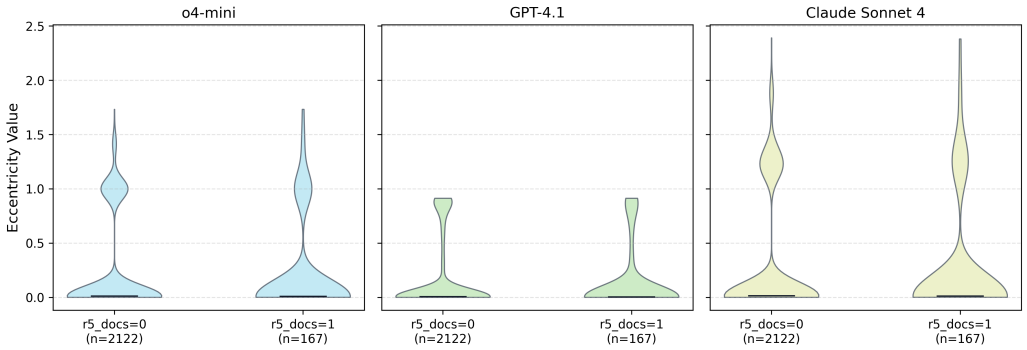


Fig. 20. Semantic eccentricity grouped by the presence of `r5-docs`.